Ghostnet for Hyperspectral Image Classification

Mercedes E. Paoletti, *Senior Member, IEEE*, Juan M. Haut, *Senior Member, IEEE*, Nuno S. Pereira, Javier Plaza, *Senior Member, IEEE*, and Antonio Plaza, *Fellow, IEEE*

***Abstract*— Hyperspectral imaging (HSI) is a competitive remote sensing technique in several fields, from Earth observation to health, robotic vision, and quality control. Each HSI scene contains hundreds of (narrow) contiguous spectral bands. The amount of data generated by HSI devices is often both a solution and a problem for a given application. Extracting information from HSI data cubes is a complex and computationally demanding problem. To tackle this challenge, convolutional neural networks (CNNs) have been widely applied to HSI classification. Despite their success, CNNs are computationally demanding algorithms with high memory requirements due to their large number of internal parameters. The recent interest in using HSI devices in mobile and embedded systems for air and spaceborne platforms turned the attention to computationally lightweight CNN architectures with good classification accuracy. In this article, we present a contribution in that direction. The proposed method combines the ghost-module architecture with a CNN-based HSI classifier to reduce the computational cost and, simultaneously, achieves an efficient classification method with high performance. Our new method is evaluated against nine standard HSI classifiers, and five improved deep-CNN architectures, over five commonly used HSI data sets for algorithm benchmarking. Conducted experiments show that the proposed method exhibits similar or better performance than the other classifiers, achieving top values in the considered performance metrics—even for very limited training sets—and, most importantly, with a fraction of the computational cost. Our novel approach for HSI classification is a strong candidate for implementation on systems with limited computational resources.**

***Index Terms*— Classification, deep learning (DL), embedded systems, hyperspectral.**

I. Introduction

HYPERSPECTRAL imaging (HSI) technology, a special case of spectral imaging, is a noninvasive technique in remote sensing particularly important when the samples under observation are to be kept intact. Moreover, the possibility of applying this technique locally, in laboratory environments, or remotely, in airborne and spaceborne platforms, makes it even more interesting for Earth observation (EO). The core of an HSI system is a specially designed sensor that collects electromagnetic radiation emitted, or reflected, by the scene under observation, at contiguous and/or noncontiguous narrow spectral bands. The multiple spatially aligned (coregistered) images collected by the sensor are stacked to form a data cube where each pixel (a vector) is a 1-D discrete spectral representation of the response at a given spatial location (see Fig. 1, wavelength λ dimension), and each 2-D layer is a spatial image representing the response at a specific wavelength band (see Fig. 1, spatial XY dimensions). The information available from an HSI data cube is, therefore, much richer than the one obtained by imaging systems that rely on limited wavelength bands on specific regions of the electromagnetic spectrum (e.g., ultraviolet, visible, and infrared). However, retrieving the necessary information from the raw HSI cubes (that allows the extraction of the spectrum measured at each pixel) requires complex data processing steps. First, raw data obtained from the sensor (level 0 data product) must be calibrated to obtain data in physical units (level 1 data product). Second, for airborne and spaceborne systems, it is necessary to compensate for the global effect of the atmosphere due to the reflection, selective absorption and emission, scattering, and transmission of radiation, using an appropriate model of radiation transfer. After the application of an atmospheric correction algorithm, the final data product (level 2) is a data cube that contains reflectance or emissivity spectra that characterizes the materials in the observed scene [1]. Currently available HSI systems generate data cubes with hundreds of wavelength bands, typically in the visible and near-infrared region of the electromagnetic spectrum, from which spectral and spatial information can be extracted for the characterization of the observed scene. The limited spatial resolution of the sensors,

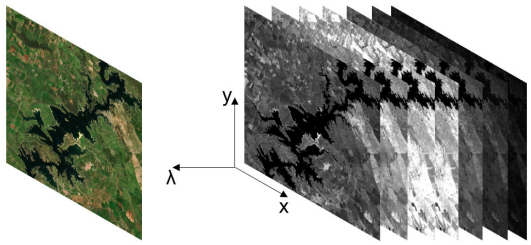


Fig. 1. True-color 2-D image created by the composition of red–blue–green channels (Left) and representation of a multiband 3-D data cube (Right). The images were obtained from the EO Browser of Sentinel-2A L2A products.

Manuscript received July 10, 2020; revised December 22, 2020; accepted January 4, 2021. Date of publication January 22, 2021; date of current version November 24, 2021. This work was supported by the Junta de Extremadura (Decreto 14/2018, de 6 de febrero, por el que se establecen las bases reguladoras de las ayudas para la realización de actividades de investigación y desarrollo tecnológico, de divulgación y de transferencia de conocimiento por los Grupos de Investigación de Extremadura) under Grant GR18060, *(Corresponding author: Mercedes E. Paoletti.)*

Mercedes E. Paoletti is with the Department of Computer Architecture, School of Computer Science and Engineering, University of Malaga, 29071 Málaga, Spain (e-mail: mpaoletti@unex.es).

Juan M. Haut is with the Department of Communication and Control Systems, National Distance Education University, 28015 Madrid, Spain (e-mail: juanmariohaut@unex.es).

Nuno S. Pereira, Javier Plaza, and Antonio Plaza are with the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, Escuela Politécnica, University of Extremadura, 10003 Cáceres, Spain (e-mail: nuno.pereira@ipbeja.pt; jplaza@unex.es; aplaza@unex.es).

Digital Object Identifier 10.1109/TGRS.2021.3050257

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

the mixing of materials at a microscopic level, and multiple scattering effects prevent a simple and direct identification of materials present in each pixel. The spectrum contained in each pixel vector of the data cube is, in fact, a mixture (overlap) of all the spectra of the pure materials (endmembers) present on the field of view of that particular pixel. The inverse problem, referred to as unmixing, consists of identifying all, or at least estimate some of the principal endmembers, their spectral signatures and the abundances in each pixel. This problem is an algorithmic and computational challenge that requires adequate approaches (see [2] and [3] for an overview of methods) and, in some cases, the inclusion of spatial information through preprocessing algorithms [4].

The possibility of identifying materials from the HSI spectral properties made this technique a powerful tool for a wide range of science and technology fields. For instance, several examples of HSI applications can be found in the fields of geology [5], landscape characterization [6], vegetation monitoring [7], [8], food quality control [9], waste management [10], ocean plastic detection [11], [12], and coastal and inland water quality monitoring [13]–[15]. In particular, for remote sensing and EO, HSI has been used for more than three decades [16], [17].

A. Deep Learning for HSI Processing

Artificial neural networks (ANNs) have been used in remote sensing image processing for more than 20 years, at first for simple tasks such as classification of land cover, cloud identification, and classification [18]. This computational model is well adapted to the nature of remote sensing data due to its capacity to learn from large collections of examples. A particular architecture of ANNs, the convolutional neural networks (CNNs), specially adapted for image recognition and classification, has shown promise in remote sensing applications due to their ability for automatically performing local feature extraction on raw data. Images are 2-D structures in which neighboring pixels are strongly correlated, thus giving the CNN a good advantage in terms of extracting local features. In this network approach, the output of scanning (convolution) an image with local receptive fields (the kernels) defines a feature map. A convolution layer may have several feature maps, with different weight vectors, to extract different features, followed by local averaging and subsampling layers, which defines the typical sequence of layer operations of this architecture [19]. The ability of these deep learning (DL) networks to extract features from high-dimensional data [20] became very attractive to the remote sense community for tasks such as land use and classification, scene classification, and object detection [21], [22]. In fact, several neural network-based approaches have been used for HSI classification with different degrees of success.

DL architectures use a large number of parameters and, therefore, require a proportionally large number of training samples to avoid problems, such as overfitting. In particular, for HSI implementations, several improvements have been proposed to overcome these difficulties, to increase computational performance and classification accuracy and to reduce the complexity of the networks [23]–[29]. On the other hand, DL algorithms are computationally intensive, require a large amount of memory, and consume a lot of energy, making these algorithms not environmentally friendly [30]. To address these issues, specific hardware such as graphic processing units (GPUs), field-programmable gate arrays (FPGAs), and dedicated chips have been developed with their performance tested on HSI classification algorithms. The different hardware solutions are targeting not only servers (e.g., for DL cloud computing), edge, and fog computing systems (associated with sensors, to decrease processing latency) but also embedded system for mobile, airborne, and spatial platforms, with limited energy, memory, and computing resources. These constraints present a new challenge in terms of DL algorithm design and are pushing the development of less hardware-demanding algorithms capable of achieving comparable results in terms of classification accuracy. In particular, for HSI applications, some hardware platforms already available exhibit promising performance [31], but there is still room for improvement from the algorithm design side.

B. Main Contributions

In order to provide an appropriate answer to the main concerns pointed above, this work proposes a new lightweight model for accurate HSI data classification. Inspired by previous models that attempt to reduce both the number of parameters and the computational cost associated with the convolution layer [32], [33], the proposed deep convolutional network exploits the advantages provided by the ghost module [34], which divides the standard convolution layer into two steps. First, a significantly lighter convolutional layer is applied to the input data, which reduces the number of filters involved to extract spectral–spatial information. Second, several computationally cheaper linear operations are applied to the extracted feature maps, in order to combine the information and reduce the high data redundancy suffered during the traditional feature extraction within the standard convolution layers. Finally, the obtained featured maps are concatenated to obtain the final output volume. As a result, the architecture greatly reduces the computational cost while retaining the overall accuracy (OA) and replacing intrinsic feature maps with simple low-cost linear transformations. Moreover, the decrease in global computational cost (compute power and memory demand), and consequently in energy consumption, has proven not only to improve the performance of deep models [35] but also to be an indispensable demand when targeting embedded systems.

To the best of our knowledge, this is the first attempt in the related literature to evaluate and measure the suitability of the ghost module in the task of addressing the inherent challenges and limitations that hyperspectral remote sensing imagery imposes when these huge data cubes are processed through DL models. Indeed, the high spectral variability and the lack of labeled samples lead to the fast degradation of deep models, which usually demands many samples to properly cover all the data variations and tend quickly to overfit. This work provides

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

a new deep convolutional model, whose performance is compared with standard and state-of-the-art DL-based methods over benchmark HSI data sets, using different configurations of training samples and input spatial sizes. The obtained results support the impressive accuracy of the method (which is quite close to current models) while significantly reducing the number of parameters involved, thus preventing model overfitting while reducing memory consumption. As a result, this article provides a novel response to the main concerns of the scientific community when dealing with DL for HSI processing.

The remainder of this article is organized as follows. The methodology is presented in Section II. In Section III, the experimental results from several HSI classifiers and the proposed method are presented and discussed. This section also includes a description of the HSI data sets and experimental settings. Finally, in Section IV, relevant conclusions are presented, namely, the adequacy of the proposed model to HSI classification and its potential for embedded system implementations.

II. Methodology

A. Convolutional Neural Networks

ANNs were first implemented as simple, fully connected, and shallow structures and then evolved to deeper and more complex architectures for tasks, such as land cover classification, data restoration, and cloud identification [18]. This computational model is well adapted to the intrinsic nature of remote sensing data due to its ability to learn from large collections of samples without prior knowledge of the data distribution. A particular architecture of ANNs, the CNN, is specially adapted for image recognition and classification [19]. Instead of fully connected hidden layers, which would be prohibitively expensive from a computational point of view for large input images (exhibiting over-parameterization that would result in high overfitting of the model), CNNs develop a local receptive field for each neuron of the hidden layer. This idea goes back to the late 1950s when the perceptron architecture was proposed by Rosenblatt (see [36], and references therein). Basically, each neuron of a convolutional layer receives inputs from the neurons on the previous layer located in a small neighborhood window. The receptive field represents the spectral–spatial extent of the connectivity of each neuron, which is defined as one of the hyperparameters of the model, and forces the extraction of local features. The set of weights (and a bias parameter) associated with a particular receptive field defines a particular filter. This is randomly initialized to some distribution (such as normal or Gaussian [37], [38]) and learned as the network is trained to detect different features within the input data, such as edges and embosses, while preserving the relationship between pixels. In this sense, the image is scanned by applying the learned filter, generating as a result a feature map. This operation is equivalent to the convolution of the image with a small kernel. In fact, the activations within a feature map indicate both the occurrence of a feature and its location and intensity within the input data. Thus, different filters generate different feature maps. The depth of the output of the convolutional layer will have the same dimension as the number of filters used in the convolution operation. By stacking several convolutional layers on top of each other, more abstract and in-depth information can be extracted from the original input data, obtaining at the end of the model an abstract data representation that has been automatically adapted to the problem posed (e.g., land cover classification, denoising, or super-resolution, among others).

With this in mind, let us consider an input HSI patch defined as $\mathbf{X} \in \mathbb{N}^{N\times N\times K}$, where the naturals N and K indicate the height/width of the image and the number of spectral bands, respectively. Following this representation, the HSI input data can be considered as a matrix of $N \times N$ vector elements (pixels), where each (i, j) pixel is computed as $\mathbf{x}_{i,j} = (x_{i,j,1}, \ldots, x_{i,j,K}) \in \mathbb{N}^K$, where $i = 1, \ldots, N$ and $j = 1, \ldots, N$ denote the spatial indices and K the number of bands on the data cube.

In addition, any convolutional layer comprises $K^{(l)}$ filters (l identifies the layer within a CNN architecture), where each one is defined as a multidimensional weight array (depending on the type of convolution, CNN1D, CNN2D, or CNN3D, respectively). For instance, considering a 2-D convolutional layer, the set of weighs is arranged as $\mathbf{W}^{(l)} \in \mathbb{R}^{M^{(l)}\times M^{(l)}\times K^{(l-1)}\times K^{(l)}}$, where $M^{(l)}$ denotes the kernel spatial height and width, $K^{(l-1)}$ the input data channels, and $K^{(l)}$ the number of filters [39], [40]. In this context, the lth convolution operation can be interpreted as the linear operation expressed by (1a), which involves the multiplication of the corresponding set of weights and the layer input volume, which can be denoted as $\mathbf{X}^{(l-1)} \in \mathbb{R}^{N^{(l-1)}\times N^{(l-1)}\times K^{(l-1)}}$, to which the corresponding bias vector $\mathbf{b}^{(l)}$ is added. As a result, an intermediate output feature map $\mathbf{Z}^{(l)} \in \mathbb{R}^{N^{(l)}\times N^{(l)}\times K^{(l)}}$ is obtained, which summarizes the locations of detected features in the input

$$\mathbf{Z}^{(l)} = \mathbf{W}^{(l)} * \mathbf{X}^{(l-1)} + \mathbf{b}^{(l)} \tag{1a}$$

$$z^{(l)}_{i,j,t} = \sum_{\hat{i},\hat{j},\hat{t}} w^{(l)}_{\hat{i},\hat{j},\hat{t},t} x^{(l-1)}_{i+\tilde{i},j+\tilde{j},\hat{t}} + b^{(l)}_t. \tag{1b}$$

In fact, as (1b) indicates, applying convolutional filters over inputs is an elementwise product between each kernel weight and the input elements that fall into the local receptive field, where $\tilde{i} = \hat{i} - \lceil N^{(l-1)}/2 \rceil$ and $\tilde{j} = \hat{j} - \lceil N^{(l-1)}/2 \rceil$ are defined as the recentered spatial indices, considering i, j, $\hat{i}$, and $\hat{j}$ as the indices along the spatial dimensions of the input/output data and the weights, respectively, and $\hat{t}$ and t the spectral indices. Fig. 2 provides a graphical explanation about the 2-D convolutional layer computation.

Finally, to learn the nonlinear relationships of the data, a nonlinear activation function $\mathcal{H}(\cdot)$ function is included, which returns the final output feature maps $\mathbf{X}^{(l)} \in \mathbb{R}^{N^{(l)}\times N^{(l)}\times K^{(l)}}$

$$\mathbf{X}^{(l)} = \mathcal{H}\left(\mathbf{Z}^{(l)}\right) \tag{2}$$

where $\mathcal{H}$ can be implemented as the smooth sigmoid and hyperbolic tangent (tanh) or the rectified linear unit (ReLU) functions, among others, that have been traditionally used in backpropagation algorithms [23], [36], [41].

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

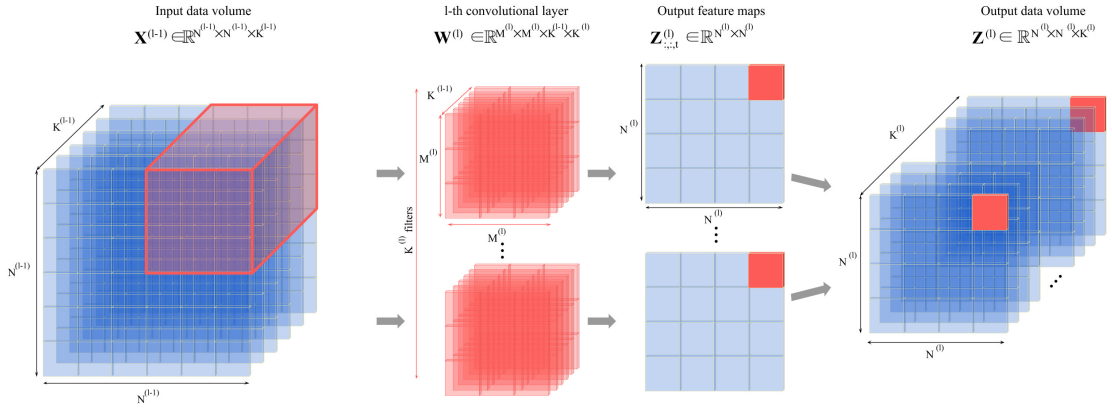


Fig. 2. Graphical visualization of a standard 2-D convolutional layer operation. Considering the lth layer, $\mathbf{W}^{(l)} \in \mathbb{R}^{M^{(l)} \times M^{(l)} \times K^{(l-1)} \times K^{(l)}}$ weights are multiplied to inputs $\mathbf{X}^{(l-1)} \in \mathbb{R}^{N^{(l-1)} \times N^{(l-1)} \times K^{(l-1)}}$, overlapping and slipping each filter through a certain stride s (i.e., step of the kernel displacement to define how the image is extended at the borders to accommodate the kernel while scanning the input volume; in this case, $s = 1$). The area emphasized in red within the input volume marks the size of the local receptive field on which the filter is applied, while the areas emphasized in red within the output volumes indicate the result obtained from applying the filter on the input data. Each filter application obtains an output array with size $N^{(l)} \times N^{(l)}$, where $N^{(l)} = \left\lfloor (N^{(l-1)} - M^{(l)} + 2p/s) \right\rfloor + 1$, considering padding p As a result, the intermediate output feature maps $\mathbf{Z}^{(l)} \in \mathbb{R}^{N^{(l)} \times N^{(l)} \times K^{(l)}}$ are obtained, as each filter is applied separately, stacking the results one on top of each other and combining them into the final volume.

It is easy to observe a large number of learnable parameters generated by a single convolutional layer. In this regard, it is worth mentioning that model HSI inputs should be generally obtained by cropping the HSI scene into two sets (one for training and one for inference stage), extracting from each pixel a neighborhood window, which is often small to avoid spatial overlapping [29]. This fact, in conjunction with the high spectral dimensionality, the high intraclass variability, and the lack of training samples to completely cover the data distribution, leads to a major problem of overfitting within deep CNN models. Moreover, the great dimensionality of the data and the large number of parameters to be trained impose severe computational and storage constraints, involving a heavy computational burden and significant memory consumption that are difficult to assume by embedded devices. With the aim of facing these limitations, great efforts have been done to reduce the dimensionality of HSI data [31], [42], [43], including the development of more representative training sample selection methodologies [23] and the implementation of lightweight models [44], [45]. However, few studies have been made to optimize the redundancy of those feature maps extracted by the deep model. It is precisely in this context that we introduce a new contribution with the objective of developing an efficient neural architecture that provides rich and intrinsic feature maps at a lower computational and storage cost.

B. Reducing Parameters and Feature Redundancy Through the Ghost Module

As pointed out above, deep CNNs often consist of a large number of convolution layers, so (1a) can be rewritten for a CNN with L convolution layers, such as

$$\mathbf{X}^{(L)} = \mathcal{F}_L(\mathcal{F}_{L-1}(\ldots \mathcal{F}_1(\mathbf{X}) \ldots)) \tag{3}$$

where each $\mathcal{F}_l = \left(\mathbf{X}^{(l-1)}, \mathbf{W}^{(l)}, \mathbf{b}^{(l)}\right)$ defines the lth convolutional layer, resulting in a massive amount of learnable parameters that raise both computing and storage costs (activation functions are omitted to simplify the nomenclature). In fact, the number of parameters and floating point operations (FLOP) involved in a CNN model can easily be calculated by (4a) and (4b), respectively [33], [34]

$$\text{Parameters: } \sum_l \left(M^{(l)}\right)^2 \times K^{(l-1)} \times K^{(l)} \tag{4a}$$

$$\text{FLOPs: } \sum_l K^{(l)} \times \left(N^{(l)}\right)^2 \times K^{(l-1)} \times \left(M^{(l)}\right)^2. \tag{4b}$$

It is noteworthy that the number of learnable parameters to be optimized is explicitly determined by the dimensions of the input $\mathbf{X}^{(l-1)}$ and output $\mathbf{X}^{(l)}$ feature maps of each convolutional layer l, where $K^{(l-1)}$ and $K^{(l)}$ are usually very large. Furthermore, it is widely known that the output volume $\mathbf{X}^{(l)}$ often contains a great amount of redundancy, comprising fairly similar features that barely contribute any information to the model, while consuming a large number of FLOPs and parameters (usually in the order of hundreds of thousands) that hamper the model performance. To overcome this, the proposed deep model aims to reduce the computational/storage resources needed for CNN models by reducing both the number of convolution filters developed and the number of redundant feature maps.

In this regard, we can assume that there is a batch of intrinsic feature maps $\tilde{\mathbf{X}}^{(l)}$, from which the output feature maps $\mathbf{X}^{(l)}$ are obtained as "ghosts" by applying some cheap transformations [34]. This batch of intrinsic feature maps, defined as $\tilde{\mathbf{X}}^{(l)} \in \mathbb{R}^{N^{(l)} \times N^{(l)} \times \tilde{K}^{(l)}}$, is generated by a primary convolution following (1a), where a set of filters denoted by $\tilde{\mathbf{W}}^{(l)} \in \mathbb{R}^{M^{(l)} \times M^{(l)} \times K^{(l-1)} \times \tilde{K}^{(l)}}$ is applied over the input layer data, with $\tilde{K}^{(l)} < K^{(l)}$. Then, to further obtain the original $K^{(l)}$ feature maps, several cheap linear operations are applied on each intrinsic feature of $\tilde{\mathbf{X}}^{(l)}$ to generate G ghost features according to the following equation:

$$\mathbf{x}^{(l)}_{:,:,t,q} = \mathbf{\Phi}_{t,q}(\tilde{\mathbf{x}}^{(l)}_{:,:,t}), \quad \forall t = 1, \ldots, \tilde{K}^{(l)}, \ \forall q = 1, \ldots, G \tag{5}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

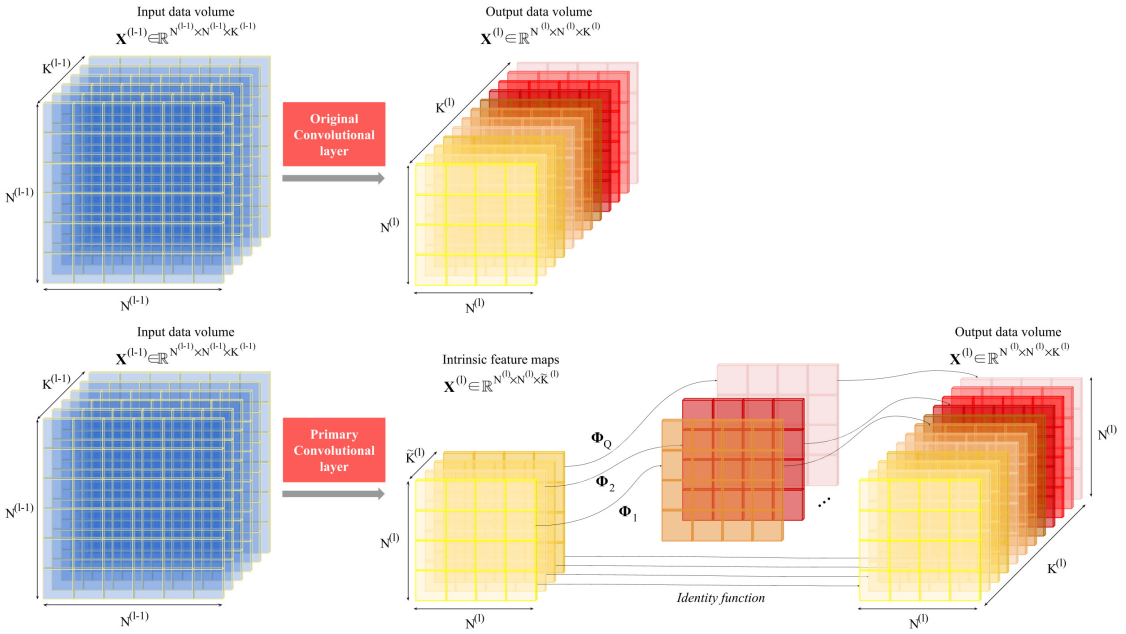


Fig. 3. Graphical visualization of traditional convolutional layer and proposed ghost module, where the original layer is divided into two stages: a much lighter convolutional layer (in the sense $\tilde{K}^{(l)} < K^{(l)}$) followed by a set of G cheap linear operations that are applied on the intrinsic feature maps. Finally, the intrinsic and ghost feature maps are concatenated to obtain the desired output volume.

where $\tilde{\mathbf{x}}^{(l)}_{:,:,t} \in \tilde{\mathbf{X}}^{(l)}$ is the tth intrinsic feature map, and $\mathbf{\Phi}_{t,q}$ defines the qth linear operation that obtains the qth ghost feature map from the tth intrinsic feature map, $\mathbf{x}^{(l)}_{:,:,t,q}$. As a result, the desired $K^{(l)} = G \cdot \tilde{K}^{(l)}$ feature maps are obtained.

Fig. 3 provides a graphical representation of the entire process, where the original convolutional layer is divided into two stages. The first one applies a lightweight (also known as primary) pointwise convolution with a much smaller number of filters, which reduces both the number of parameters and FLOPs given by (4), extracting the intrinsic feature maps $\tilde{\mathbf{X}}^{(l)}$. The second one applies the G cheap operations over the extracted $\tilde{\mathbf{X}}^{(l)}$ to obtain the final output feature maps $\mathbf{X}^{(l)}$, operating on each channel and reducing the computation cost too. It must be noted that several linear operations are, in fact, identity functions in order to preserve the intrinsic feature maps, where the remaining Q's are different linear operations implemented as 3×3 linear kernels.

We can estimate the number of parameters and FLOPs that are theoretically consumed by the ghost module, in order to perform a theoretical comparison between this module and the traditional convolution. Considering a standard convolution with kernel size $K^{(l)} \times M^{(l)} \times M^{(l)} \times K^{(l-1)}$, which is applied over the input feature maps $\mathbf{X}^{(l-1)}$ with size $N \times N \times K^{(l-1)}$ (in principle, we consider the spatial dimensions as a constant), to obtain the output feature maps $\mathbf{X}^{(l)}$ (we omitted the intermediate steps to simplify the procedure), with the corresponding size $N \times N \times K^{(l)}$, we can replace it by a ghost module to obtain the same number of feature maps. First, the lighter primary layer with size $\tilde{K}^{(l)} \times \tilde{M}^{(l)} \times \tilde{M}^{(l)} \times K^{(l-1)}$ is applied. In our implementation, this layer is implemented as a pointwise convolution $\tilde{M}^{(l)} = 1$, while $\tilde{K}^{(l)} = K^{(l)}/2$. It obtains the output feature maps $\mathbf{Y}^{(l)}$ with size $N \times N \times \tilde{K}^{(l)}$. Then, the linear operations are applied over $\mathbf{Y}^{(l)}$. These are implemented as a grouped convolution [46] where each $M^{(l)} \times M^{(l)}$ filter is applied to each channel of $\mathbf{Y}^{(l)}$, obtaining the output $\mathbf{Z}^{(l)}$ with size $N \times N \times \tilde{K}^{(l)}$. Finally, $\mathbf{Y}^{(l)}$ and $\mathbf{Z}^{(l)}$ are concatenated to obtain the final $\mathbf{X}^{(l)}$ with size $N \times N \times K^{(l)}$. Following (4), we can approximate the number of parameters and FLOPs as follows:

$$\begin{aligned}\text{Parameters: } &\left(\tilde{K}^{(l)} \times K^{(l-1)}\right) + \left(\tilde{K}^{(l)} \times \left(M^{(l)}\right)^2\right) \\ &= \left(\frac{K^{(l)}}{2} \times K^{(l-1)}\right) + \left(\frac{K^{(l)}}{2} \times \left(M^{(l)}\right)^2\right) \qquad (6a)\\ \text{FLOPs: } &\left(\tilde{K}^{(l)} \times N^2 \times K^{(l-1)}\right) + \left(\tilde{K}^{(l)} \times N^2 \times \left(M^{(l)}\right)^2\right) \\ &= \left(\frac{K^{(l)}}{2} \times N^2 \times K^{(l-1)}\right) + \left(\frac{K^{(l)}}{2} \times N^2 \times \left(M^{(l)}\right)^2\right). \qquad (6b)\end{aligned}$$

Let us consider a simplified practical example, i.e., a standard $48 \times 3 \times 3 \times 16$ convolutional layer that is applied over an $11 \times 11 \times 16$ input volume. The obtained output volume will be $11 \times 11 \times 48$. In this sense, the standard convolution contains $48 \cdot 3 \cdot 3 \cdot 16 = 6912$ parameters and involves $48 \cdot 11^2 \cdot 16 \cdot 3^2 = 836\,352$ FLOPs. We can replace the standard convolution by a ghost module composed by the $24 \times 1 \times 1 \times 16$ primary layer and the $24 \times 3 \times 3 \times 24$ grouped convolution layer, which together contains $(24 \cdot 16) + (24 \cdot 3^2) = 600$ parameters, i.e., about 91.39%[1] of parameters have been removed from the

[1]It must be noted that this percentage, 91.39%, has been obtained taking into account a simple rule of three, where 6912 represents 100% of parameters within the standard convolution layer and $((6912 - 600) \cdot 100)/(6912) = (6312 \cdot 100)/(6912) = 91.39\%$ is the total amount of parameters removed.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

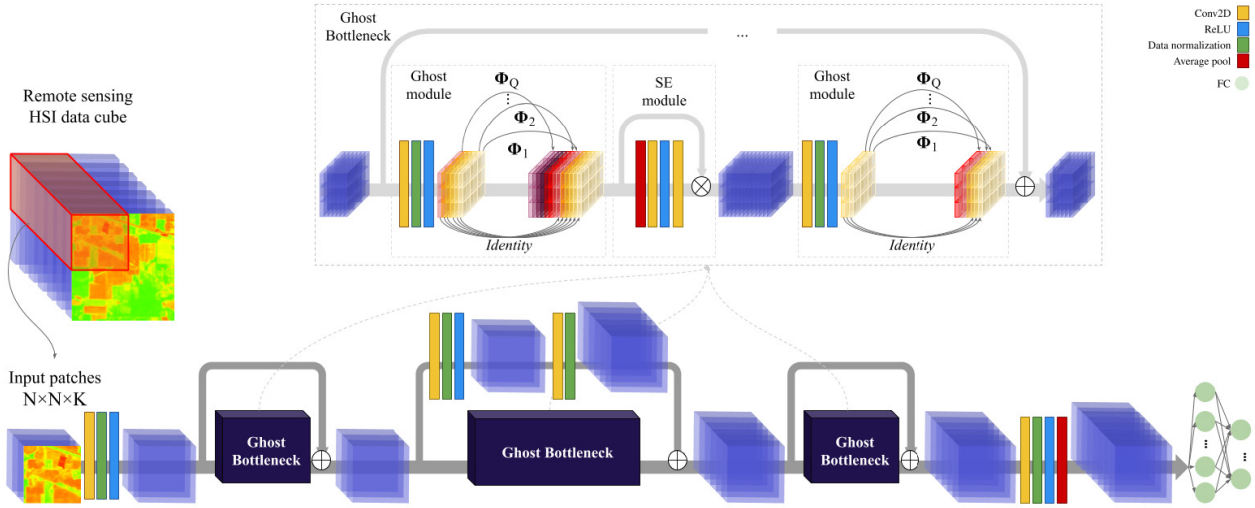


Fig. 4. Graphical overview of the proposed network architecture. With the exception of the first one, which keeps the number of channels constant throughout the whole block, ghost bottlenecks expand and reduce the number of channels through their ghost modules, applying a channel-based attention mechanism by including of SE modules. Moreover, depending on whether the bottleneck maintains the same number of channels at its input and output, an identity function within the shortcut connection is implemented or not.

layer, which represents a significant reduction in the number of operations that are carried out. Indeed, the proposed module involves only $(24 \cdot 11^2 \cdot 16) + (24 \cdot 11^2 \cdot 3^2) = 72600$ FLOPs. As we can observe, in theory, the ghost module is able to reduce approximately 11 times the number of parameters and FLOPs consumed by the standard convolutional layer.

C. Proposed Model Architecture for HSI Classification

With these concepts in mind, a new and simpler ghost-based architecture has been developed to perform HSI data classification in an effective and efficient way. As we can observe in Fig. 4, the proposed network implements a simple stem unit, which is composed of convolution-norm-activation layers, to reduce the spectral complexity of the input data. Then, three ghost bottlenecks are implemented. In this sense, our proposed model drastically reduces the number of bottlenecks used for the classification of HSI data in order to avoid not only model overfitting but also data degradation and the vanishing of gradients during the forward and backward steps, respectively, which are quite characteristic of very deep models when processing this kind of remote sensing data. Furthermore, these blocks are inspired by residual bottlenecks [26], [47], taking advantage of shortcut connections, which relieves the so-called declining-accuracy phenomenon when considering significantly deep networks. In this regard, the residual connections provide a direct and simple way to exploit more efficiently the features that might, otherwise, remain uncovered. Each bottleneck comprises two stacked ghost modules (see Fig. 3). Each ghost module is composed of a primary pointwise convolutional layer, which extracts the intrinsic feature maps by processing the input channels, and a grouped convolutional layer (denoted as *Conv.), which comprises as many groups as input channels, to force each linear kernel to been applied on one channel of the intrinsic feature maps. As a result, each filter combines the spatial information in the corresponding channel. The resulting output volume of the grouped convolutional layer is concatenated to the output volume of the primary pointwise convolutional layer and sent to the next block. Between each ghost module, a squeeze-and-excitation (SE) block [48] is implemented to enhance the channelwise feature responses by combining both types of "primary" and "grouped" features. The SE block comprises an average pooling layer and two pointwise convolutions. Except for the first one, the ghost bottlenecks increase the number of channels in the first ghost module and reduce it again in the second one, mimicking the behavior of a residual bottleneck [47]. In this sense, the first module triples the number of channels, while the SE-module compresses and extends them again to combine the information across the channels so that, finally, the second ghost module reduces the number of channels to the desired one. In addition, the second bottleneck increases the number of features at its output, so it needs to adjust the size of its input feature maps by several convolutions in the shortcut connection before performing the final sum. In the end, the final convolutional-pooling block collects all the features and vectorizes them before sending the resulting output to the classifier, which is implemented as a two-layer fully connected (FC) multilayer perceptron (MLP). Table I provides the implementation details for each layer. It should be noted that all average pools have been implemented using the adaptive average pool, which adapts itself to the input sizes in order to vectorize the multidimensional input array into a 1-D array.

Finally, the proposed model for HSI data classification has been trained in about 500 epochs, considering the stochastic gradient descent (SGD) as an optimizer, with a learning rate of 0.1 and a batch size of 100.

1) Analyzing Model Configuration: Regarding the architecture configuration, we have conducted an in-depth appraisal of the strengths and weaknesses of the selected configuration. In particular, we have compared the performance of the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

TABLE I

IMPLEMENTED ARCHITECTURE FOR HSI DATA CLASSIFICATION. K DEFINES THE NUMBER OF SPECTRAL BANDS AND C THE CORRESPONDING NUMBER OF LAND COVER CLASSES. *CONV IS THE LINEAR KERNEL. PERCENTAGES NEXT TO ACTIVATION FUNCTIONS ARE THE AMOUNT OF DROPOUT

ID	Type	Kernel/neurons	Str./Pad.	Norm.	Act. funct.
1	**Stem unit**				
	Conv.	$16 \times 3 \times 3 \times K$	1/No	Yes	ReLU (20%)
2	**Ghost-Bottleneck**				
	Ghost-Module				
	Conv.	$8 \times 1 \times 1 \times 16$	1/No	Yes	ReLU
	*Conv.	$8 \times 3 \times 3 \times 8$	1/Yes	Yes	ReLU
	SE-Module				
	AvgPool.				
	Conv.	$4 \times 1 \times 1 \times 16$	1/No	No	ReLU
	Conv.	$16 \times 1 \times 1 \times 4$	1/No	No	-
	Ghost-Module				
	Conv.	$8 \times 1 \times 1 \times 16$	1/No	Yes	-
	*Conv.	$8 \times 3 \times 3 \times 8$	1/Yes	Yes	-
3	**Ghost-Bottleneck**				
	Ghost-Module				
	Conv.	$24 \times 1 \times 1 \times 16$	1/No	Yes	ReLU
	*Conv.	$24 \times 3 \times 3 \times 24$	1/Yes	Yes	ReLU
	SE-Module				
	AvgPool.				
	Conv.	$12 \times 1 \times 1 \times 48$	1/No	No	ReLU
	Conv.	$48 \times 1 \times 1 \times 12$	1/No	No	-
	Ghost-Module				
	Conv.	$12 \times 1 \times 1 \times 48$	1/No	Yes	-
	*Conv.	$12 \times 3 \times 3 \times 12$	1/Yes	Yes	-
	Shortcut-Connection				
	Conv.	$16 \times 3 \times 3 \times 16$	1/Yes	Yes	ReLU
	*Conv.	$24 \times 1 \times 1 \times 16$	1/No	Yes	-
4	**Ghost-Bottleneck**				
	Ghost-Module				
	Conv.	$36 \times 1 \times 1 \times 24$	1/No	Yes	ReLU
	*Conv.	$36 \times 3 \times 3 \times 36$	1/Yes	Yes	ReLU
	SE-Module				
	AvgPool.				
	Conv.	$20 \times 1 \times 1 \times 72$	1/No	No	ReLU
	Conv.	$72 \times 1 \times 1 \times 20$	1/No	No	-
	Ghost-Module				
	Conv.	$12 \times 1 \times 1 \times 72$	1/No	Yes	-
	*Conv.	$12 \times 3 \times 3 \times 12$	1/Yes	Yes	-
5	Conv.	$72 \times 1 \times 1 \times 24$	1/No	Yes	ReLU
6	AvgPool.				
7	FC	216	-	Yes	ReLU (20%)
8	FC	C	-	No	Softmax

proposed ghost-based bottleneck (composed by ghost-SE-ghost modules) against its residual counterpart proposed by He *et al.* [47], which comprises three convolution layers with 1×1, 3×3, and 1×1 kernels. Indeed, we attempt to contrast it with its convolution-based counterpart. The comparisons have been conducted over Indian Pines (IP) and Pavia University (PU) scenes with disjoint training and testing samples. We have evaluated both the accuracy and computing performances by taking into account the following measurements: OA, average accuracy (AA), and kappa coefficient have been taken into account to evaluate the quality and reliability of the classification procedure, while the runtimes and number of parameters and multiply-and-accumulate (MAC) operations have been considered to assess the computational performance. Table II provides the obtained results. It can be seen that the proposed model always achieves higher accuracy than the standard residual architecture, particularly in complex HSI scenes, such as IP, where the low spatial resolution produces highly mixed spectral signatures. Conversely, the PU scene is quite simple, as its signatures (nine different land cover types instead of 16) are clearer, and the spatial information helps to separate the different classes, so there is a slight overfitting effect.

TABLE II

COMPARISON BETWEEN PROPOSED GHOST-BASED AND STANDARD RESIDUAL BOTTLENECK ARCHITECTURES FOR HSI CLASSIFICATION

	Indian Pines		Pavia University	
	ResNet	**Proposed**	**ResNet**	**Proposed**
OA(%)	86.75	**88.31**	92.44	**92.83**
AA(%)	77.17	**78.77**	**91.49**	91.37
K(x100)	84.92	**86.70**	89.68	**90.20**
Parameters (K.)	67	**60**	51	**44**
ParametersB (K.)	16	**9**	16	**9**
MACs (K.)	3818	**2921**	2685	**1788**
Runtime (s.)	**285**	390	**240**	320

Regarding the computational performance, the proposed model noticeably reduces the number of parameters, in particular taking into account only the parameters comprised by each bottleneck (denoted as ParametersB), where the ghost one has 1.78 fewer parameters than the residual counterpart. As a result, about 43.75% of parameters have been removed without impairing the reliability of the model. This has a clear impact on the number of MAC operations. Indeed, the proposed model conducts fewer MACs than the residual model.

Finally, regarding the runtimes, it must be noticed that the current runtimes have been obtained after 500 epochs. However, Fig. 5 conducts a deeper analysis about the model convergence, in which we can observe how the proposed model reaches a quite high and stable OA and loss during the training stage around the 50 epochs. Moreover, it can achieve fairly acceptable results in the first five to ten epochs, needing less than the residual model, which stabilizes from 10 to 15 epochs.

In addition, focusing on the most challenging data set, i.e., IP scene, we have conducted an ablation study to assess the performance of the proposed ghost-SE-ghost bottleneck (denoted as Proposed) with shortcut connection architecture against the ghost-SE-ghost without shortcut connection (denoted as NoSC, i.e., we have removed only the shortcut connection only), the ghost–ghost with shortcut connection (denoted as NoSE, i.e., we have removed only the SE module), and the ghost–ghost without shortcut (denoted as *NoSE-NoSC*, i.e., we have removed both the SE module and the shortcut connection) architectures. Table III provides the obtained results. As we can observe, despite the fact that 5M more parameters are included than the lighter NoSE-NoSC model, the accuracy achieved by the proposed model is significantly greater (between 3% and 5% points of OA). In this sense, we conduct a tradeoff study between model complexity and accuracy and determine the ghost-SE-ghost with shortcut connection architecture as the most optimal one to be evaluated in Section III.

III. EXPERIMENTAL RESULTS

A. Hyperspectral Data Sets

The experiments were conducted over five hyperspectral data sets generally used for the purpose of performance

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

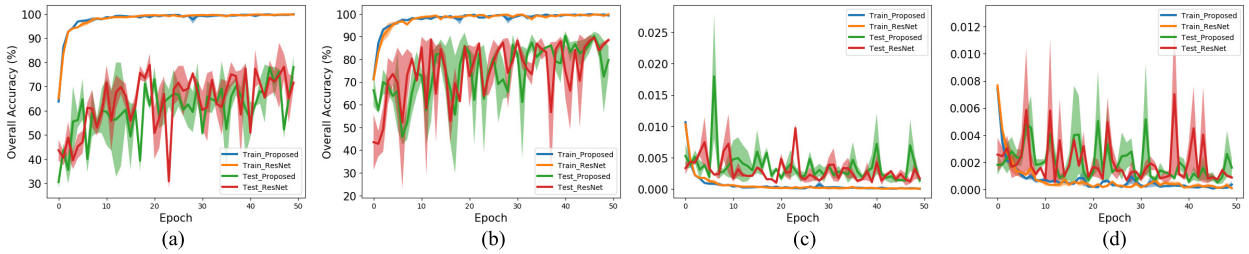
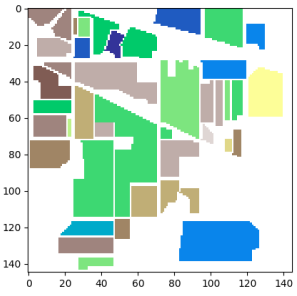


Fig. 5. OA and loss evolution considering IP and PU scenes with disjoint training/test samples. (a) IP OA. (b) PU OA. (c) IP loss. (d) PU loss.

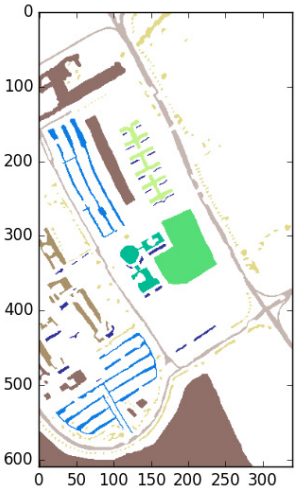
TABLE III
ABLATION STUDY OVER IP SCENE

	Proposed	NoSC	NoSE	NoSE-NoSC
OA	**88.31**	85.91	85.42	83.15
AA	**78.77**	75.4	75.66	72.22
K (x 100)	**86.70**	83.96	83.4	80.81
Parameters (K.)	60	59	56	**55**



Color	Land cover type	Samples
	Background	10776
	Alfalfa	46
	Corn-notill	1428
	Corn-min	830
	Corn	237
	Grass/Pasture	483
	Grass/Trees	730
	Grass/pasture-mowed	28
	Hay-windrowed	478
	Oats	20
	Soybeans-notill	972
	Soybeans-min	2455
	Soybean-clean	593
	Wheat	205
	Woods	1265
	Bldg-Grass-Tree-Drives	386
	Stone-steel towers	93
	Total samples	21025

Fig. 6. Ground truth of the IP scene.

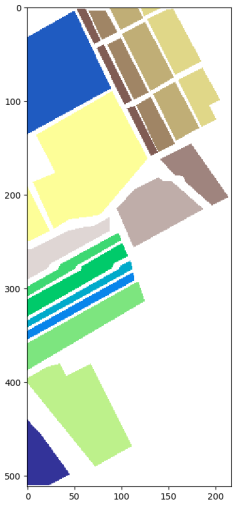


Color	Land cover type	Samples
	Background	164624
	Asphalt	6631
	Meadows	18649
	Gravel	2099
	Trees	3064
	Painted metal sheets	1345
	Bare Soil	5029
	Bitumen	1330
	Self-Blocking Bricks	3682
	Shadows	947
	Total samples	207400

Fig. 7. Ground truth of the PU scene.

evaluation of HSI algorithms. Figs. 6–9 present the available ground truth information. A more detailed description of the data sets is presented in the following.

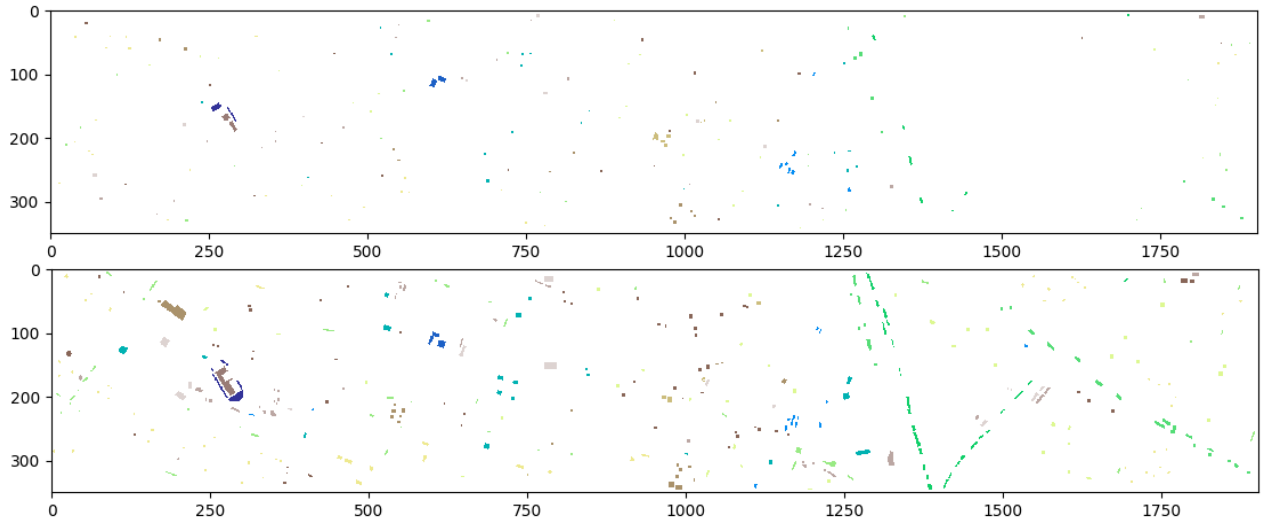
1) The first data set is known as IP. It was gathered by the Airborne Visible Infra-Red Imaging Spectrometer (AVIRIS) instrument [49] during a flying campaign over the IP test site in Northwestern Indiana in 1992. The captured area is characterized by several crops and irregular forest and pasture patches. It comprises 145×145 pixels, each of which has 224 spectral reflectance bands covering the wavelengths from 400 to 2500 nm. We remove the bands 104-108, 150-163, and 220 (water absorption and null bands) and keep 200 bands in our experiments. This scene has 16 different ground-truth classes (see Fig. 6).
2) The second data set is the PU scene. It was acquired by the Reflective Optics Spectrographic Imaging System (ROSIS) instrument [50] during a flight campaign over Pavia city, Northern Italy. In this sense, it is characterized by being an urban area, with areas of buildings, roads, and parking lots. In particular, the PU scene has 610×340 pixels, and its spatial resolution is 1.3 m. The original Pavia data set contains 115 bands in the spectral region of 0.43–0.86 μm. We remove the water absorption bands and retain 103 bands in our experiments. The number of classes in this scene is 9 (see Fig. 7).
3) The third data set is **Salinas Valley** (SV). It was gathered by the AVIRIS instrument too. The collected area is characterized by regular fields of different crops. It has 512×217 pixels and covers SV in California. We remove the water absorption bands 108-112, 154-167, and 224 and keep 204 bands in our experiments. This scene contains 16 classes (see Fig. 8).



Color	Land-cover type	Samples
	Background	56975
	Brocoli-green-weeds-1	2009
	Brocoli-green-weeds-2	3726
	Fallow	1976
	Fallow-rough-plow	1394
	Fallow-smooth	2678
	Stubble	3959
	Celery	3579
	Grapes-untrained	11271
	Soil-vinyard-develop	6203
	Corn-senesced-green-weeds	3278
	Lettuce-romaine-4wk	1068
	Lettuce-romaine-5wk	1927
	Lettuce-romaine-6wk	916
	Lettuce-romaine-7wk	1070
	Vinyard-untrained	7268
	Vinyard-vertical-trellis	1807
	Total samples	111104

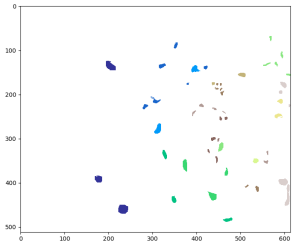
Fig. 8. Ground truth of the SV scene.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.



Color	Land cover type	Samples train	Samples test	Color	Land cover type	Samples train	Samples test
	Background	649816			Grass-healthy	198	1053
	Grass-stressed	190	1064		Grass-synthetic	192	505
	Tree	188	1056		Soil	186	1056
	Water	182	143		Residential	196	1072
	Commercial	191	1053		Road	193	1059
	Highway	191	1036		Railway	181	1054
	Parking-lot1	192	1041		Parking-lot2	184	285
	Tennis-court	181	247		Running-track	187	473
Total samples 2832|12197							

Fig. 9. Ground truth of the Houston scene.



Color	Land cover type	Samples
	Background	309157
	Scrub	761
	Willow-swamp	243
	CP-hammock	256
	Slash-pine	252
	Oak/Broadleaf	161
	Hardwood	229
	Swap	105
	Graminoid-marsh	431
	Spartina-marsh	520
	Cattail-marsh	404
	Salt-marsh	419
	Mud-flats	503
	Water	927
	Total samples	314368

Fig. 10. Ground truth of the KSC scene.

4) The fourth data set used in experiments is the **Kennedy Space Center** (KSC). As IP and SV, it was acquired by the AVIRIS sensor. The observed area corresponds to a miscellaneous region of Florida, which was captured in 1996. After removing those corrupted bands, 176 bands have been considered, with 512×614 pixels. The spectral range comprises the 400-2500 nm, with 20-m spatial resolution. The scene contains 13 different land cover classes (see Fig. 10).
5) The fifth data set is **Houston University** (HU) [51], which was acquired by the Compact Airborne Spectrographic Imager (CASI) sensor [52] over the HU campus on June 2012, collecting spectral information from an urban area. This scene has 114 bands and 349×1905 pixels with wavelengths ranging from 380 to 1050 nm. It comprises 15 ground-truth classes (see Fig. 9).

B. Experimental Settings

We have designed three experiments with the purpose of evaluating the performance of the proposed method in terms of HSI classification.

1) First, classification results are measured for three data sets (IP, PU, and SV), considering different percentages of the available labeled training samples (3%, 5%, and 10%) and the size of the input spatial patches (11×11, 13×13, and 15×15).
2) Second, for comparison purposes, nine classifiers have been considered, four of which are traditional machine learning methods, such as nonlinear support vector machine, employing radial basis function kernel (SVM) [53], random forest (RF) [54], multinomial logistic regression (MLR) [55], and a shallow neural network known as MLP [56]. In addition, three different recurrent neural networks (RNN) have been considered, in particular, the vanilla RNN [29], the long short-term memory (LSTM)-based RNN [57], and the gated-recurrent-unit (GRU)-based RNN [58]. Finally, two standard CNNs have been compared: the spectral (1-D) CNN (CNN1D) [59] and the spatial CNN

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

CLASSIFICATION RESULTS (IN PERCENTAGE) OBTAINED BY THE PROPOSED METHOD FOR THE IP, PU, AND SV SCENES USING DIFFERENT WINDOW SIZES AND PERCENTAGES OF LABELED TRAINING SAMPLES

Training size	Windows size	INDIAN PINES			PAVIA UNIVERSITY			SALINAS VALLEY		
		OA	AA	Kappa(x100)	OA	AA	Kappa(x100)	OA	AA	Kappa(x100)
3	11×11	89.88 ± 1.47	85.11 ± 1.99	88.44 ± 1.7	99.64 ± 0.14	99.41 ± 0.23	99.52 ± 0.19	99.27 ± 0.06	99.37 ± 0.1	99.19 ± 0.07
5		95.53 ± 1.06	93.42 ± 0.48	94.9 ± 1.21	99.76 ± 0.07	99.47 ± 0.18	99.68 ± 0.09	99.24 ± 0.34	99.44 ± 0.2	99.15 ± 0.38
10		98.47 ± 0.51	97.26 ± 1.55	98.25 ± 0.58	99.93 ± 0.01	99.88 ± 0.03	99.91 ± 0.02	99.53 ± 0.33	99.53 ± 0.31	99.48 ± 0.37
3	13×13	90.85 ± 1.21	87.14 ± 3.91	89.56 ± 1.39	99.34 ± 0.22	98.88 ± 0.28	99.13 ± 0.29	99.6 ± 0.11	99.58 ± 0.1	99.55 ± 0.12
5		96.01 ± 0.35	93.61 ± 1.96	95.45 ± 0.41	99.71 ± 0.08	99.45 ± 0.15	99.62 ± 0.1	99.64 ± 0.17	99.62 ± 0.24	99.6 ± 0.18
10		98.59 ± 0.15	97.43 ± 0.4	98.39 ± 0.17	99.95 ± 0.02	99.91 ± 0.02	99.93 ± 0.03	99.82 ± 0.15	99.76 ± 0.24	99.8 ± 0.17
3	15×15	92.0 ± 1.53	87.33 ± 3.34	90.88 ± 1.76	99.44 ± 0.12	98.89 ± 0.17	99.26 ± 0.15	99.57 ± 0.32	99.61 ± 0.31	99.52 ± 0.36
5		96.3 ± 0.58	93.98 ± 2.43	95.78 ± 0.65	99.75 ± 0.06	99.38 ± 0.19	99.67 ± 0.08	99.66 ± 0.23	99.56 ± 0.35	99.62 ± 0.26
10		98.62 ± 0.19	98.01 ± 0.42	98.43 ± 0.22	99.94 ± 0.01	99.83 ± 0.06	99.91 ± 0.02	99.96 ± 0.01	99.96 ± 0.01	99.96 ± 0.01

with 2-D kernel (CNN2D) [60]. With the exception of the CNN2D, considered classifiers are traditional HSI spectral (or pixelwise) methods, whereas CNN2D is a spatial classifier with a 2-D kernel, where the number of spectral bands has been reduced to one by applying principal component analysis (PCA).

The classification accuracy of each HSI algorithm was evaluated by three quantitative metrics generally accepted for this purpose [29]: OA, the AA, and the Cohen's kappa (K) coefficient [61]. In this sense, the first one computes the ratio of correctly classified HSI pixels and the number of samples, while the second one obtains the mean of the classification accuracy of all classes. Finally, the third measurement provides the reliability of agreement between the obtained classification map and the original ground-truth map. In addition, the number of parameters and the runtimes are provided to evaluate the computational performance of the proposed method. Moreover, we have computed the number of MAC instructions conducted. In this experiment, IP, PU, and HU data sets have been considered with fixed and disjoint training/test samples and an input spatial patch size of 11×11, in order to avoid the characteristic overlapping between training and testing samples from random selection.

3) The last experiment compares the proposed method with five state-of-the-art CNN-based deep architectures over three data sets (IP, PU, and KSC): the spectral–spatial residual network (SSRN) [62], the spectral–spatial RN with pyramidal-bottleneck blocks (P-RNs) [26], the densely connected RN (DenseNet) [63], the spectral–spatial dual-path network (DPN) [64], and the capsule network (CapsNet) [27]. Furthermore, for evaluating the dependence of the OA results with the size of the spatial input patch, four window sizes were used: 5×5, 7×7, 9×9, and 11×11.

The implementation of the algorithms used in this work has been developed and tested on a hardware environment with an X Generation Intel Core i9-9940X processor, which contains 19.25M of cache memory and up to 4.40 GHz (the number of cores can vary between 14-core/28-way multitask processing), installed over a Gigabyte X299 Aorus, 128 GB of DDR4 RAM. Also, an NVIDIA Titan RTX GPU with 24-GB GDDR6 of video memory and 4608 cores has been used.

C. Experiments and Discussion

1) Experiment 1*:* This experiment evaluates the dependence of the classification performance of the proposed method on the size of the training sets and spatial input patches. For that purpose, we considered 3%, 5%, and 10% of the available labeled samples for each HSI scene and three windows sizes. Table IV presents the average values and standard deviations for the metrics: OA, AA, and Kappa, after five Monte Carlo runs. Considering the three data sets, the value of K is within the range of near-perfect classification. In particular, for the PU and SV data sets, we have $K > 99\%$ in all cases. For the IP data set, K varies between 88.44% and 98.43% and never reaches 99%. This data set corresponds to a scene with higher spectral mixing, and it would probably be necessary to increase the size of training samples to reach a value of 99%. Again, for the PU and SV data sets, the values of OA and AA are greater than 99% for all configurations, except in the case of SV with 3% training and window sizes of 13×13 and 15×15, for which the AA is 98.88% and 98.89%, respectively.

For the IP data set, the performance decreases considerably with small training sets, especially when using only 3% of the available training samples, for all the window sizes considered in the experiment. However, when 5% of the training samples are used, the values of OA and AA are greater than 95.5% and 93.4%, respectively, for all window sizes. The trend of improvement in classification accuracy with the increase of training size or spatial input is clear for all data sets: a wider window and/or a bigger training set generally result in higher values for OA, AA, and K and lower values of the associated standard deviation (uncertainty). When averaged over the window sizes, as the number of training samples increases, the values of OA, AA, and K exhibit an improvement of $+7.65$, $+11.04$, and $+8.73$, respectively, for the IP data set, $+0.47$, $+0.81$, and $+0.61$, respectively, for the PU data set, and $+0.29$, $+0.23$, and $+0.33$, respectively, for the SV data set. For the standard deviation associated with each metric, there is a clear trend of decrease as the sizes of the training set and the input spatial windows increase.

2) Experiment 2*:* The results of this experiment over the three considered data sets (IP, PU, and HU) are presented in Tables V–VII. For each data set class (first column), the classification results (in percentage) for the corresponding method are displayed (next columns). The last column shows the results for the proposed method. The bottom three rows correspond to the values of the previously defined metrics:

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

TABLE V

CLASSIFICATION RESULTS (IN PERCENTAGE) OBTAINED BY DIFFERENT TECHNIQUES FOR THE IP SCENE, USING FIXED TRAINING AND TEST SETS AVAILABLE FOR THESE DATA SETS AT HTTP://DASE.GRSS-IEEE.ORG

Class	RF	MLR	SVM	MLP	RNN	LSTM	GRU	CNN1D	CNN2D	Proposed
0	37.6	68.0	**96.0**	67.2	58.4	93.6	88.0	86.4	63.2	82.4
1	51.7	78.07	80.74	84.92	74.58	82.99	80.09	82.37	42.79	**89.19**
2	44.16	59.41	70.79	69.9	63.81	64.95	64.21	**76.93**	35.79	73.42
3	27.88	25.25	51.52	45.25	35.96	48.69	49.7	47.47	26.67	**54.95**
4	79.71	88.32	87.59	87.23	85.33	88.1	87.74	88.32	46.5	**89.64**
5	95.76	96.89	96.61	96.95	94.8	96.84	**97.29**	96.95	86.72	94.46
6	20.0	50.0	**100.0**	100.0	0.0	10.0	0.0	30.0	20.0	0.0
7	**100.0**	99.2	98.8	98.96	97.92	99.6	99.28	99.92	97.84	99.52
8	10.0	40.0	70.0	76.0	46.0	74.0	68.0	90.0	50.0	**92.0**
9	8.59	56.1	81.91	78.93	74.35	80.52	84.85	76.5	42.82	**93.36**
10	89.43	81.65	87.51	81.78	81.65	78.31	81.99	83.81	79.23	**89.88**
11	26.52	68.44	80.5	78.01	69.5	76.74	78.51	80.64	33.48	**80.85**
12	88.0	96.25	93.75	97.75	96.75	96.5	96.75	97.25	**98.0**	94.25
13	91.93	89.98	91.93	93.98	90.28	94.31	92.55	93.21	81.91	**98.13**
14	38.38	82.83	78.79	**85.86**	75.56	84.44	82.42	80.4	54.34	49.7
15	93.64	93.18	88.64	83.64	91.82	90.45	**94.09**	90.45	90.0	78.64
OA	65.68	78.16	85.08	83.99	79.69	82.89	83.57	84.7	62.23	**88.31**
AA	56.46	73.35	**84.69**	82.9	71.04	78.75	77.84	81.29	59.33	78.77
K(x100)	59.86	75.0	82.98	81.77	76.84	80.53	81.3	82.56	56.58	**86.7**
Parameters (K.)	-	-	-	**31**	217	255	243	73	627	60
MACs (K.)	-	-	-	62.02	8.45	409.70	409.70	321.16	143286.75	2921.08
Runtime (s.)	**4**	31	5	35	1354	331	327	58	311	390

TABLE VI

CLASSIFICATION RESULTS (IN PERCENTAGE) OBTAINED BY DIFFERENT TECHNIQUES FOR THE PU SCENE USING FIXED TRAINING AND TEST SETS AVAILABLE FOR THESE DATA SETS AT HTTP://DASE.GRSS-IEEE.ORG

Class	RF	MLR	SVM	MLP	RNN	LSTM	GRU	CNN1D	CNN2D	Proposed
0	79.5	77.68	82.23	86.12	82.65	80.63	79.95	89.08	66.74	**92.05**
1	55.11	58.79	65.81	76.48	67.26	77.26	80.42	88.56	84.44	**97.76**
2	45.41	67.22	66.72	66.56	60.79	58.1	68.28	71.89	47.0	**82.04**
3	**98.71**	74.28	97.77	93.23	89.75	96.0	93.07	93.41	96.17	96.13
4	99.16	98.9	99.37	99.26	99.21	99.34	99.37	99.37	**99.39**	99.14
5	78.68	**93.53**	91.62	89.39	86.77	69.11	71.5	86.55	61.94	71.82
6	80.22	85.1	87.36	86.87	84.14	84.75	82.55	**88.36**	70.6	88.01
7	90.93	87.57	90.46	90.89	88.61	90.24	85.76	87.68	89.16	**97.22**
8	97.69	**99.25**	93.71	97.53	99.09	95.87	96.96	89.33	92.83	98.16
OA	70.08	72.23	77.8	82.77	76.99	79.62	81.05	88.25	78.87	**92.83**
AA	80.6	82.48	86.12	87.37	84.25	83.48	84.21	88.25	78.7	**91.37**
K(x100)	62.93	65.44	72.06	77.74	70.78	73.46	75.15	84.39	72.06	**90.2**
Parameters (K.)	-	-	-	**9**	72	110	97	34	505	44
MACs (K.)	-	-	-	17.60	8.45	118.71	118.71	146.75	88722.80	1788.16
Runtime (s.)	12	6	**5**	110	1765	625	630	146	1460	320

TABLE VII

CLASSIFICATION RESULTS (IN PERCENTAGE) OBTAINED BY DIFFERENT TECHNIQUES FOR THE HU SCENE USING FIXED TRAINING AND TEST SETS AVAILABLE FOR THESE DATA SETS AT HTTP://DASE.GRSS-IEEE.ORG

Class	RF	MLR	SVM	MLP	RNN	LSTM	GRU	CNN1D	CNN2D	Proposed
0	82.47	82.26	82.34	81.56	82.26	**82.79**	82.75	82.26	81.27	81.69
1	83.36	82.48	83.36	82.01	82.65	81.02	82.16	88.23	**90.98**	87.23
2	97.94	99.8	99.8	99.64	99.8	99.68	**99.92**	99.72	90.89	98.76
3	91.72	98.3	**98.96**	88.09	94.22	91.04	96.08	97.25	84.98	94.27
4	96.74	97.44	98.77	97.41	98.01	97.75	96.74	98.66	**99.96**	99.07
5	**99.16**	94.41	97.9	94.55	95.1	96.36	98.18	96.92	95.38	95.42
6	75.34	73.41	77.43	74.74	81.51	79.42	79.2	83.69	82.13	**84.63**
7	33.12	63.82	60.3	67.33	40.27	40.36	54.15	76.35	72.55	**86.91**
8	69.56	70.27	76.77	68.22	77.05	78.41	77.77	**79.81**	71.41	77.4
9	44.02	55.6	61.29	49.48	47.36	48.51	50.06	56.54	**67.37**	65.22
10	69.68	74.19	80.55	78.48	76.38	76.89	79.94	86.45	88.41	**93.86**
11	54.12	70.47	79.92	83.15	78.98	82.8	85.28	89.16	91.18	**96.13**
12	59.58	67.72	70.88	69.61	70.46	68.98	71.02	74.88	77.96	**81.35**
13	99.43	98.79	**100.0**	99.43	100.0	99.6	99.68	99.35	98.14	99.73
14	97.34	95.56	96.41	98.31	98.22	98.18	98.14	98.27	97.63	**98.58**
OA	73.01	78.98	81.86	79.33	78.38	78.36	80.61	85.36	84.27	**87.87**
AA	76.91	81.63	84.31	82.13	81.49	81.45	83.41	87.17	86.02	**89.35**
K(x100)	71.0	77.31	80.43	77.72	76.66	76.71	79.04	84.12	82.92	**86.84**
Parameters (K.)	-	-	-	**2**	51	89	76	9	427	37
MACs (K.)	-	-	-	**4.09**	8.45	76.89	76.89	32.95	53286.54	1054.62
Runtime (s.)	2	11	2	45	921	328	341	66	183	160

OA, AA, and K. Globally, the proposed method achieves the best results. In all but one case, it shows the best values for the metrics. In particular, K presents values corresponding to near-perfect classification in all cases. In terms of performance over the classes, the proposed method presents better or equal classification results for 50.0%, 44.4%, and 40.0% of the classes of IP, PU, and HU data sets, respectively. Compared with the second best spectral classifier (SVM for the IP data set and CNN1D for the other data sets), the average performance improvement is +3.4, +2.7, and +4.1 for OA, AA, and K metrics, respectively.

Regarding the computational complexity, the runtimes and the number of parameters and MACs have been compared for different models. As it can be observed, spectral classifiers (MLP, RNN, LSTM, GRU, and CNN1D) exhibit the smallest number of parameters as they do not apply multidimensional weight arrays to the input data, which is usually reshaped into 2-D arrays. On the contrary, CNN2D and the proposed model apply more complex kernels to the input volume data, and therefore, they need to adjust more weights than their spectral counterparts. However, the proposed model contains considerably fewer parameters than the standard CNN2D. For instance, focusing on the IP scene, the proposed model contains 10.45 times fewer parameters than the CNN2D, while, in PU and HU, the ratio is 11.48 and 11.54, respectively. This has a clear impact on the number of MAC instructions conducted by each model; particularly, the proposed model reduces approximately 49.05, 49.62, and 50.53 times the number of conducted instructions for IP, PU, and HU scenes, respectively. Regarding the runtimes, Tables V–VII provide the execution times considering the training and validation stages. On the one hand, it must be taken into account that RF, MLR, SVM, MLP, RNN, LST, GRU, CNN1D, and CNN2D implementations have been extracted from the repository provided by [29]. In this sense, these methods follow the specified configurations and have been developed into Keras. On the other hand, the proposed model has been developed in Pytorch following the configuration described in Section II-II-C. Given this heterogeneous environment in terms of batch sizes and the number of epochs (for instance, our model runs about 500 epochs, while the CNN1D and CNN2D models run 300), it is hard to evaluate fairly the runtimes of each model. In addition, related to the number of epochs, we have established 500 as an estimated number of epochs; however, it must be taken into account that our proposed network can converge at a minimum (i.e., achieving a high and stable accuracy) in 50 epochs approximately for some data sets, such as the IP as we pointed out in Section II-C1. In this sense, we can observe that, in the tenth part of the runtime, our model can achieve an accuracy result very close to that expressed by the OA, AA, and Kappa values actually collected. For instance, in the IP data set, the model is converging at 39 s approximately, while, in PU and HU scenes, it can converge after 32 and 16 s, respectively, converging significantly faster than the CNN2D and even the CNN1D. Therefore, a deeper and more dedicated analysis is required to conduct a strict and fair comparison.

Figs. 11–13 illustrate some of the classification maps associated with the results presented in Tables V–VII, respectively. It is clear that noisy classification images are associated with spectral classifiers since they do not consider the spatial dimension of the data on the pixel prediction process. In contrast, the CNN2D spatial classifier and the proposed method produce well-defined images in terms of border delineation. However, the former tends to alter the shapes of some objects and introduces artifacts in class boundaries, caused by the fact that pixel prediction is determined by spatial information,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

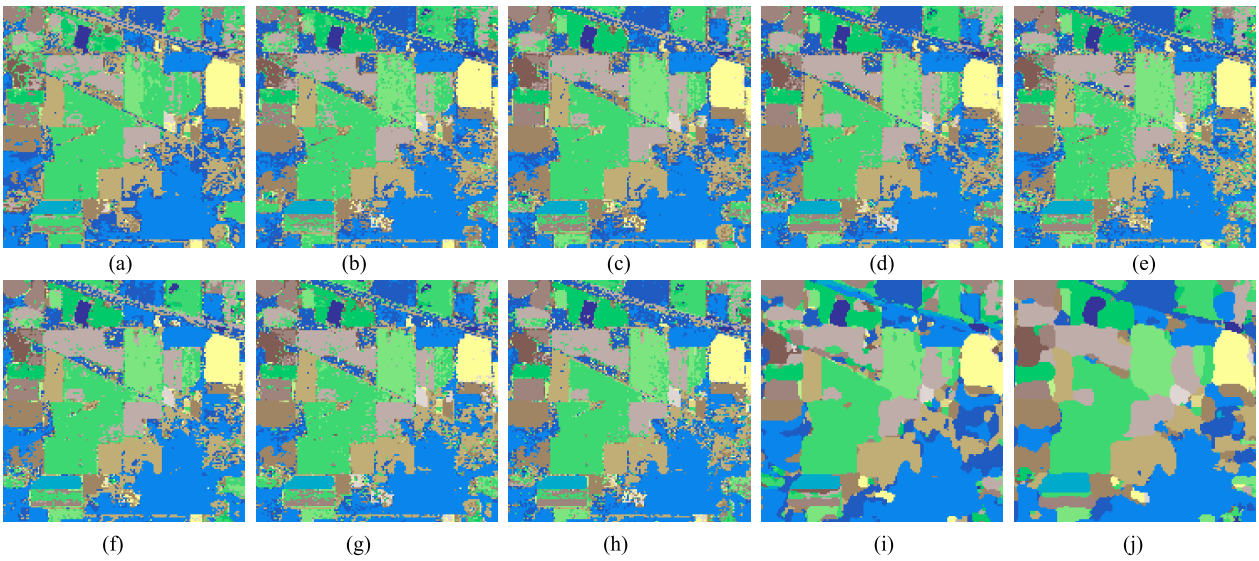


Fig. 11. Classification maps obtained for the IP scene by different classifiers (see Table V). Corresponding OA values are shown in brackets, while the best result is highlighted in bold font. (a) RF (65.68%). (b) MLR (78.16%). (c) SVM (85.08%). (d) MLP (83.99%). (e) RNN (79.69%). (f) LSTM (83.57%). (g) GRU (82.89%). (h) CNN1D (84.70%). (i) CNN2D (62.23%). (j) **Proposed (88.31%)**.

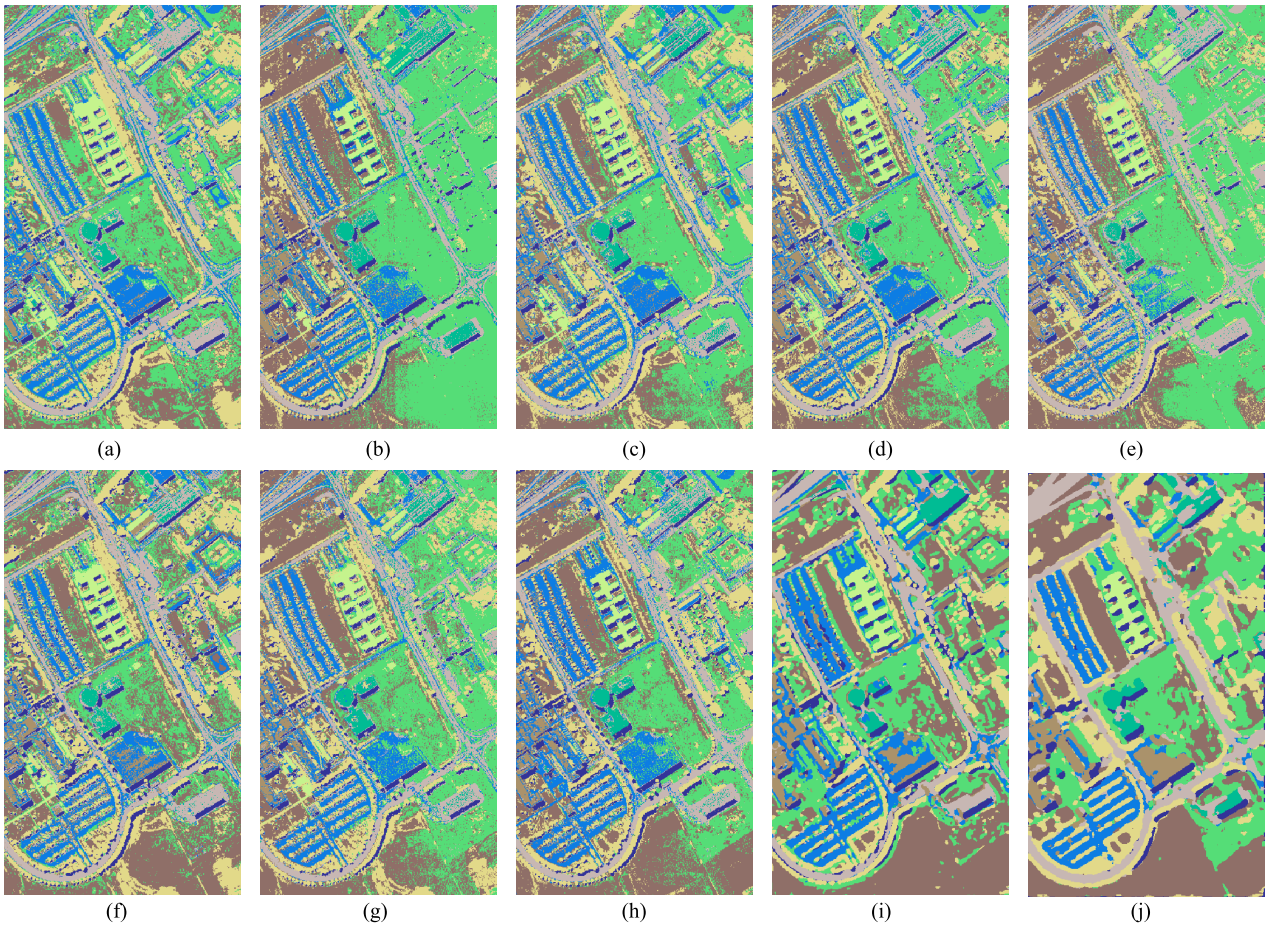


Fig. 12. Classification maps obtained for the PU scene by different classifiers (see Table VI). Corresponding OA values are shown in brackets, while the best result is highlighted in bold font. (a) RF (70.08%). (b) MLR (72.23%). (c) SVM (77.80%). (d) MLP (82.77%). (e) RNN (76.99%). (f) LSTM (81.05%). (g) GRU (79.62%). (h) CNN1D (88.25%). (i) CNN2D (78.87%). (j) **Proposed (92.83%)**.

which, in turn, increases the sensitivity of the method to the spatial size of the input window. The proposed method not only produces cleaner and more defined classification maps but also achieves higher overall classification accuracies. It is worth noting that, when the unlabeled areas are considered (those not covered by ground truth), the proposed method

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

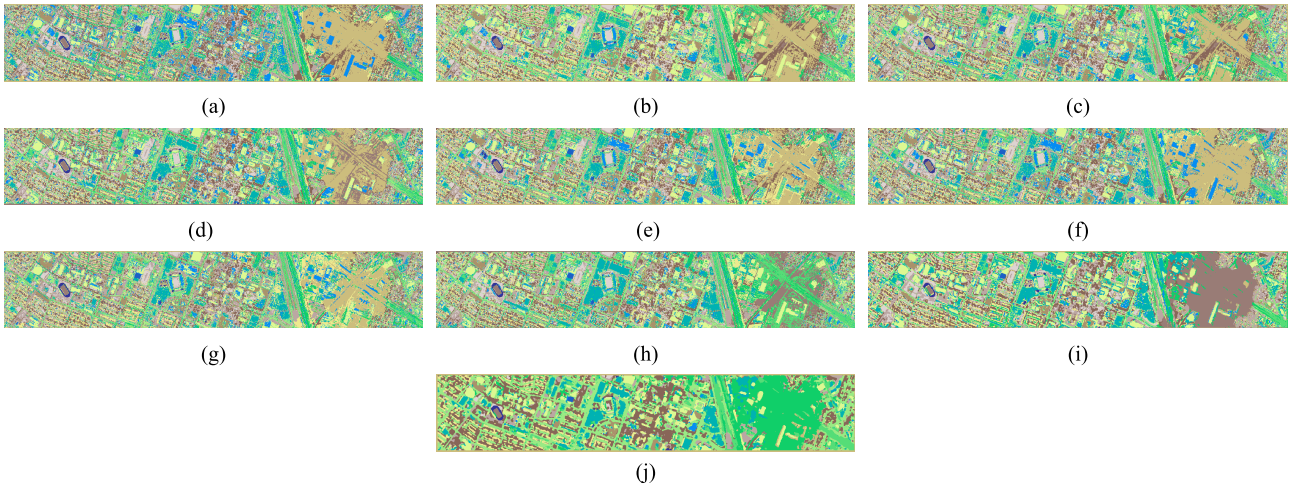


Fig. 13. Classification maps obtained for the HU scene by different classifiers (see Table VII). Corresponding OA values are shown in brackets, while the best result is highlighted in bold font. (a) RF (73.01%). (b) MLR (78.98%). (c) SVM (81.86%). (d) MLP (79.33%). (e) RNN (78.38%). (f) LSTM (80.61%). (g) GRU (78.36%). (h) CNN1D (85.36%). (i) CNN2D (84.27%). (j) **Proposed (87.87%)**.

TABLE VIII

OBTAINED OA VALUES (%) ACHIEVED BY CONSIDERED DL CLASSIFIERS WHEN USING DIFFERENT INPUT SPATIAL SIZES, COUPLED WITH PARAMETER ESTIMATION, IN ORDER TO PROVIDE AN OVERVIEW OF THE IMPLEMENTED ARCHITECTURES

	IP dataset					
Spatial Size	**SSRN**	**P-RN**	**DenseNet**	**DPN**	**CapsNet**	**Proposed**
5×5	92.83 ±0.66	**98.80** ±0.10	97.85 ±0.28	97.53 ±0.15	97.79 ±0.40	98.05 ±0.32
7×7	97.81 ±0.34	99.26 ±0.06	99.24 ±0.14	99.29 ±0.06	**99.30** ±0.11	99.09 ±0.41
9×9	98.68 ±0.29	99.64 ±0.08	99.58 ±0.09	99.64 ±0.10	**99.67** ±0.06	99.50 ±0.26
11×11	98.70 ±0.21	**99.82** ±0.07	99.74 ±0.08	99.67 ±0.06	99.74 ±0.09	99.65 ±0.20
	PU dataset					
Spatial Size	**SSRN**	**P-RN**	**DenseNet**	**DPN**	**CapsNet**	**Proposed**
5×5	98.72 ±0.17	**99.52** ±0.05	99.13 ±0.08	99.21 ±0.11	99.13 ±0.08	99.45 ±0.12
7×7	99.54 ±0.11	**99.81** ±0.09	99.71 ±0.10	99.70 ±0.07	99.75 ±0.03	99.51 ±0.44
9×9	99.57 ±0.54	99.79 ±0.11	99.73 ±0.15	**99.88** ±0.04	99.73 ±0.10	99.72 ±0.47
11×11	99.79 ±0.08	99.92 ±0.02	99.93 ±0.03	**99.94** ±0.03	99.93 ±0.02	99.92 ±0.14
	KSC dataset					
Spatial Size	**SSRN**	**P-RN**	**DenseNet**	**DPN**	**CapsNet**	**Proposed**
5×5	98.72 ±0.17	**99.52** ±0.05	99.13 ±0.08	99.21 ±0.11	99.13 ±0.08	96.26 ±0.61
7×7	99.54 ±0.11	**99.81** ±0.09	99.71 ±0.10	99.70 ±0.07	99.75 ±0.03	98.35 ±0.67
9×9	99.57 ±0.54	99.79 ±0.11	99.73 ±0.15	**99.88** ±0.04	99.73 ±0.10	98.93 ±0.94
11×11	99.79 ±0.08	99.92 ±0.02	99.93 ±0.03	**99.94** ±0.03	99.93 ±0.02	99.71 ±0.15
Parameters	360K.	2.4M.	1.7M.	370K.	9.0M.	**44K.-60K.**

provides classification results that appear to be more consistent (with fewer outliers and artifacts) than those provided by the other classifiers. This is an important feature related to the generalization capability of the method.

3) Experiment 3: This experiment compares the OA of the proposed method with that achieved by five conv-based approaches over the IP, PU, and KSC data sets, considering multiple input spatial sizes. The results in Table VIII show that the proposed method compares well with the other methods and allows three important observations to be highlighted: 1) the proposed method achieves an accuracy higher than 99% for the majority of experiment configurations (in eight out of 12 cases), even with small input spatial sizes, and it is never lower than 96%; 2) the difference in accuracy to the best method is between 0.02% and 3.26% (with an average of 0.65%); and 3) the standard deviation values of the proposed method are compared with those observed for the other methods. It is worth mentioning that, for the IP data set (in which the spectral mixing is higher), the proposed method exhibits performance similar to that of the other methods. In fact, for the case of the lowest spatial size (5×5), it presents an improvement of $+5.22\%$, compared with SSRN, and half the standard deviation. On the other hand, compared with P-RN (the method with the best performance in this case), the difference in OA is as small as 0.75%. To give a global overview of the classification performance, Table IX presents the variation on the averaged OA achieved by the proposed method and, in the bottom row, the ratio (in percentage) between the numbers of estimated parameters for the proposed method (worst case) and those for each of the compared methods. In 60% of the cases, the absolute variation of the OA is insignificant ($\leq 0.30\%$), and in 90% of the cases, it is

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
VARIATION OF THE AVERAGED OA ACHIEVED BY THE PROPOSED METHOD AND THE RATIO (IN PERCENTAGE) BETWEEN THE ESTIMATED PARAMETERS FOR THE PROPOSED METHOD AND THE COMPARED METHODS

Dataset	SSRN	P-RN	DenseNet	DPN	CapsNet
IP	2.07	-0.31	-0.03	0.04	-0.05
PU	0.25	-0.11	0.03	-0.03	0.02
KSC	-1.09	-1.48	-1.31	-1.37	-1.31
Ratio (%)	16.67	2.50	3.53	16.22	0.67

below 1.5%. There is even a 2.07% increase of performance in one case (compared with SSRN, for the IP data set). This is a remarkable result, particularly if the parameters ratio is taken into account. In fact, the proposed method achieves similar (or better) results with a smaller fraction of the number of parameters estimated for the other methods. In the worst case, the proposed method needs around 17% of the number of parameters estimated for SSRN, and (in the best case) 0.67% of the number of parameters estimated for CapsNet. This is a very important observation that reveals the potential of the proposed method for implementation in architectures where compute power and memory size are critical, such as mobile and embedded devices [65].

In summary, the previous experiments showed that the proposed method achieves a very good classification accuracy for all the scenarios tested and with different configurations of training samples and spatial input window sizes. Compared with standard HSI classifiers, our method exhibits similar (or better) performance in a significant number of classes for each data set and presents the best values for the metrics in all but one case. Finally, a comparison with improved convolutional-based classifiers revealed that the proposed method achieves similar (and in some cases better) results with a fraction of the computational cost needed by those methods.

IV. CONCLUSION

HSI technologies are becoming more attractive for applications in a broad range of fields, as the cost of hardware decreases and the computational power increases. Some of the most important and interesting HSI applications involve DL-based methods to tackle the challenge of extracting information out (big) data cubes that, in turn, provide very detailed knowledge of the observed scene. These methods, although efficient, are compute-intensive and memory demanding. In addition, they require a significant amount of labeled data sets to avoid problems, such as overfitting. On the other hand, the recent interest in mobile and embedded systems (e.g., for airborne and spaceborne platforms) with HSI (real-time) classification ability has been promoting the development of computationally lightweight DL methods, suitable to fit the constraints imposed by the limited computational power and available memory on such devices. However, those methods must exhibit good performance in terms of classification accuracy, uncertainty, requirements in terms of training sets sizes, and runtime. This work aims to contribute to this challenge by presenting a new computationally efficient classification method that has been tested with commonly used HSI benchmark data sets and compared with a variety of standard HSI classifiers and improved CNN-based methods. The obtained results revealed an excellent performance of our newly proposed method with all the considered scenes, outperforming some state-of-the-art architectures in a significant number of cases and, most importantly, with only a fraction of their complexity. These results suggest that our newly proposed architecture is a potential candidate for embedded systems and other low-power devices. In the future, we will conduct extensive tests analyzing the performance–power tradeoff of our newly proposed method in different architectures.

REFERENCES

[1] D. G. Manolakis, R. B. Lockwood, and T. B. Cooley, *Hyperspectral Imaging Remote Sensing: Physics, Sensors, and Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2016.

[2] J. M. Bioucas-Dias *et al.*, "Hyperspectral unmixing overview: Geometrical, statistical, and sparse regression-based approaches," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 5, no. 2, pp. 354–379, Apr. 2012.

[3] J. Plaza, E. M. T. Hendrix, I. García, G. Martín, and A. Plaza, "On endmember identification in hyperspectral images without pure pixels: A comparison of algorithms," *J. Math. Imag. Vis.*, vol. 42, nos. 2–3, pp. 163–175, Feb. 2012.

[4] J. Delgado, G. Martin, J. Plaza, L. I. Jimenez, and A. Plaza, "Fast spatial preprocessing for spectral unmixing of hyperspectral data on graphics processing units," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 2, pp. 952–961, Feb. 2016.

[5] F. D. van der Meer *et al.*, "Multi-and hyperspectral geologic remote sensing: A review," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 14, no. 1, pp. 112–128, Feb. 2012.

[6] J. B. Adams and A. R. Gillespie, *Remote Sensing of Landscapes With Spectral Images: A Physical Modeling Approach*. Cambridge, U.K.: Cambridge Univ. Press, Jan. 2006.

[7] J. Im and J. R. Jensen, "Hyperspectral remote sensing of vegetation," *Geography Compass*, vol. 2, no. 6, pp. 1943–1961, Nov. 2008.

[8] P. Mishra, M. S. M. Asaari, A. Herrero-Langreo, S. Lohumi, B. Diezma, and P. Scheunders, "Close range hyperspectral imaging of plants: A review," *Biosyst. Eng.*, vol. 164, pp. 49–67, Dec. 2017.

[9] T.-T. Pan, E. Chyngyz, D.-W. Sun, J. Paliwal, and H. Pu, "Pathogenetic process monitoring and early detection of pear black spot disease caused by alternaria alternata using hyperspectral imaging," *Postharvest Biol. Technol.*, vol. 154, pp. 96–104, Aug. 2019.

[10] C. Signoret, A.-S. Caro-Bretelle, J.-M. Lopez-Cuesta, P. Ienny, and D. Perrin, "MIR spectral characterization of plastic to enable discrimination in an industrial recycling context: II. Specific case of polyolefins," *Waste Manage.*, vol. 98, pp. 160–172, Oct. 2019.

[11] L. Goddijn-Murphy, S. Peters, E. van Sebille, N. A. James, and S. Gibb, "Concept for a hyperspectral remote sensing algorithm for floating marine macro plastics," *Mar. Pollut. Bull.*, vol. 126, pp. 255–262, Jan. 2018.

[12] S. P. Garaba *et al.*, "Sensing ocean plastics with an airborne hyperspectral shortwave infrared imager," *Environ. Sci. Technol.*, vol. 52, no. 20, p. 11699–11707, Sep. 2018.

[13] W. J. Moses *et al.*, "Estimation of chlorophyll—A concentration in turbid productive waters using airborne hyperspectral data," *Water Res.*, vol. 46, no. 4, pp. 993–1004, Mar. 2012.

[14] R. M. Kudela, S. L. Palacios, D. C. Austerberry, E. K. Accorsi, L. S. Guild, and J. Torres-Perez, "Application of hyperspectral remote sensing to cyanobacterial blooms in inland waters," *Remote Sens. Environ.*, vol. 167, pp. 196–205, Sep. 2015.

[15] S. Bagheri, *Hyperspectral Remote Sensing of Nearshore Water Quality* (SpringerBriefs in Environmental Science). Cham, Switzerland: Springer, 2017.

[16] A. F. H. Goetz, G. Vane, J. E. Solomon, and B. N. Rock, "Imaging spectrometry for Earth remote sensing," *Science*, vol. 228, no. 4704, pp. 1147–1153, Jun. 1985.

[17] A. F. H. Goetz, "Three decades of hyperspectral remote sensing of the Earth: A personal view," *Remote Sens. Environ.*, vol. 113, pp. S5–S16, Sep. 2009.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

[18] P. M. Atkinson and A. R. L. Tatnall, "Introduction neural networks in remote sensing," *Int. J. Remote Sens.*, vol. 18, no. 4, pp. 699–709, Mar. 1997.

[19] Y. LeCun and Y. Bengio, *Convolutional Networks for Images, Speech, and Time Series*, M. A. Arbib and Y. Bengio, Eds. Cambridge, MA, USA: MIT Press, 1998.

[20] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.

[21] A. E. Maxwell, T. A. Warner, and F. Fang, "Implementation of machine-learning classification in remote sensing: An applied review," *Int. J. Remote Sens.*, vol. 39, no. 9, pp. 2784–2817, May 2018.

[22] L. Ma, Y. Liu, X. Zhang, Y. Ye, G. Yin, and B. A. Johnson, "Deep learning in remote sensing applications: A meta-analysis and review," *ISPRS J. Photogramm. Remote Sens.*, vol. 152, pp. 166–177, Jun. 2019.

[23] J. M. Haut, M. E. Paoletti, J. Plaza, J. Li, and A. Plaza, "Active learning with convolutional neural networks for hyperspectral image classification using a new Bayesian approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 11, pp. 6440–6461, Nov. 2018.

[24] Y. Luo, J. Zou, C. Yao, X. Zhao, T. Li, and G. Bai, "HSI-CNN: A novel convolution neural network for hyperspectral image," in *Proc. Int. Conf. Audio, Lang. Image Process. (ICALIP)*, Jul. 2018, pp. 464–469.

[25] H. Zhang, Y. Li, Y. Jiang, P. Wang, Q. Shen, and C. Shen, "Hyperspectral classification based on lightweight 3-D-CNN with transfer learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 8, pp. 5813–5828, Aug. 2019.

[26] M. E. Paoletti, J. M. Haut, R. Fernandez-Beltran, J. Plaza, A. J. Plaza, and F. Pla, "Deep pyramidal residual networks for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 740–754, Feb. 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8445697/

[27] M. E. Paoletti *et al.*, "Capsule networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2145–2160, Apr. 2019.

[28] N. He *et al.*, "Feature extraction with multiscale covariance maps for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 755–769, Feb. 2019.

[29] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "Deep learning classifiers for hyperspectral imaging: A review," *ISPRS J. Photogramm. Remote Sens.*, vol. 158, pp. 279–317, Dec. 2019.

[30] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," Jun. 2019, *arXiv:1906.02243*. [Online]. Available: http://arxiv.org/abs/1906.02243

[31] J. M. Haut, S. Bernabe, M. E. Paoletti, R. Fernandez-Beltran, A. Plaza, and J. Plaza, "Low–high-power consumption architectures for deep-learning models applied to hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 5, pp. 776–780, May 2019.

[32] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1251–1258.

[33] B. Wu *et al.*, "Shift: A zero FLOP, zero parameter alternative to spatial convolutions," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 9127–9135.

[34] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "GhostNet: More features from cheap operations," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1580–1589.

[35] M. E. Paoletti, J. M. Haut, X. Tao, J. Plaza, and A. Plaza, "FLOP-reduction through memory allocations within CNN for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, early access, Sep. 29, 2020, doi: 10.1109/TGRS.2020.3024730.

[36] R. Rojas, *Neural Networks: A Systematic Introduction*. Berlin, Germany: Springer-Verlag, 1996.

[37] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th Int. Conf. Artif. Intell. Statist.*, 2010, pp. 249–256.

[38] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1026–1034.

[39] C.-C.-J. Kuo, "Understanding convolutional neural networks with a mathematical model," *J. Vis. Commun. Image Represent.*, vol. 41, pp. 406–413, Nov. 2016.

[40] T. Wiatowski and H. Bolcskei, "A mathematical theory of deep convolutional neural networks for feature extraction," *IEEE Trans. Inf. Theory*, vol. 64, no. 3, pp. 1845–1866, Mar. 2018.

[41] K. Hara, D. Saito, and H. Shouno, "Analysis of function of rectified linear unit used in deep learning," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2015, pp. 1–8.

[42] D. Fernandez, C. Gonzalez, D. Mozos, and S. Lopez, "FPGA implementation of the principal component analysis algorithm for dimensionality reduction of hyperspectral images," *J. Real-Time Image Process.*, vol. 16, no. 5, pp. 1395–1406, Oct. 2019.

[43] M. Diaz *et al.*, "Real-time hyperspectral image compression onto embedded GPUs," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 8, pp. 2792–2809, Aug. 2019.

[44] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*. [Online]. Available: http://arxiv.org/abs/1704.04861

[45] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 116–131.

[46] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017.

[47] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[48] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7132–7141.

[49] G. Vane, R. O. Green, T. G. Chrien, H. T. Enmark, E. G. Hansen, and W. M. Porter, "The airborne visible/infrared imaging spectrometer (AVIRIS)," *Remote Sens. Environ.*, vol. 44, nos. 2–3, pp. 127–143, 1993.

[50] B. Kunkel, F. Blechinger, R. Lutz, R. Doerffer, H. van der Piepen, and M. Schroder, "ROSIS (Reflective Optics System Imaging Spectrometer)—A candidate instrument for polar platform missions," *Proc. SPIE*, vol. 0868, pp. 134–141, Apr. 1988.

[51] X. Xu, J. Li, and A. Plaza, "Fusion of hyperspectral and LiDAR data using morphological component analysis," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2016, pp. 3575–3578.

[52] S. Babey and C. Anger, "A compact airborne spectrographic imager (CASI)," in *Proc. Quant. Remote Sens., Econ. Tool Nineties*, vol. 1, 1989, pp. 1028–1031.

[53] B. Waske, S. van der Linden, J. A. Benediktsson, A. Rabe, and P. Hostert, "Sensitivity of support vector machines to random feature selection in classification of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 7, pp. 2880–2889, Jul. 2010.

[54] J. Ham, Y. Chen, M. M. Crawford, and J. Ghosh, "Investigation of the random forest framework for classification of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 492–501, Mar. 2005.

[55] J. M. Haut, M. E. Paoletti, A. Paz-Gallardo, J. Plaza, and A. Plaza, "Cloud implementation of logistic regression for hyperspectral image classification," in *Proc. 17th Int. Conf. Comput. Math. Methods Sci. Eng. (CMMSE)*, 2017, pp. 1063–2321.

[56] R. Collobert and S. Bengio, "Links between perceptrons, MLPs and SVMs," in *Proc. 21st Int. Conf. Mach. Learn. (ICML)*. New York, NY, USA: ACM, 2004, pp. 177–184. [Online]. Available: http://portal.acm.org/citation.cfm?doid=1015330.1015415

[57] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[58] K. Cho, B. van Merrienboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder-decoder approaches," 2014, *arXiv:1409.1259*. [Online]. Available: http://arxiv.org/abs/1409.1259

[59] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, Jul. 2015, Art. no. 258619.

[60] W. Zhao, Z. Guo, J. Yue, X. Zhang, and L. Luo, "On combining multiscale deep learning features for the classification of hyperspectral remote sensing imagery," *Int. J. Remote Sens.*, vol. 36, no. 13, pp. 3368–3379, Jul. 2015.

[61] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, Apr. 1960, doi: 10.1177/001316446002000104.

[62] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[63] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "Deep & dense convolutional neural network for hyperspectral image classification," *Remote Sens.*, vol. 10, no. 9, p. 1454, Sep. 2018. [Online]. Available: http://www.mdpi.com/2072-4292/10/9/1454

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

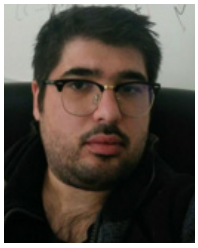
[64] X. Kang, B. Zhuo, and P. Duan, "Dual-path network-based hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 3, pp. 447–451, Mar. 2019.
[65] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, "A survey of model compression and acceleration for deep neural networks," 2017, *arXiv:1710.09282*. [Online]. Available: http://arxiv.org/abs/1710.09282



Mercedes E. Paoletti (Senior Member, IEEE) received the B.Sc. and M.Sc. degrees in computer engineering from the University of Extremadura, Cáceres, Spain, in 2014 and 2016, respectively, and the Ph.D. degree, with a University Teacher Training Programme from the Spanish Ministry of Education, from the Hyperspectral Computing Laboratory (HyperComp), Department of Technology of Computers and Communications, University of Extremadura in 2020.

She is also a Researcher with the Department of Computer Architecture, University of Malaga, Málaga, Spain. Her research interests include remote sensing and analysis of very high spectral resolution with the focus on deep learning and high-performance computing.

Dr. Paoletti was a recipient of the 2019 Outstanding Paper Award recognition in the WHISPERS 2019 Congress and the Outstanding Ph.D. Award at the University of Extremadura in 2020. She has been a Reviewer of the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING and IEEE GEOSCIENCE AND REMOTE SENSING LETTERS, in which she was recognized as one of the best reviewers of 2019.



Juan M. Haut (Senior Member, IEEE) received the B.Sc. and M.Sc. degrees in computer engineering and the Ph.D. degree in information technology, with a University Teacher Training Programme from the Spanish Ministry of Education, from the Hyperspectral Computing Laboratory (HyperComp), Department of Technology of Computers and Communications, University of Extremadura, Cáceres, Spain, in 2011, 2014, and 2019, respectively.

He was a Researcher Member with the HyperComp. He is an Associate Professor with the Department of Communication and Control Systems, National Distance Education University, Madrid, Spain. His research interests include remote sensing data processing and high-dimensional data analysis, applying machine (deep) learning, and cloud computing approach. In this sense, he has authored/coauthored more than 30 JCR journal articles (19 in IEEE journals) and 20 peer-reviewed conference proceeding papers.

Dr. Haut was a recipient of the Outstanding Ph.D. Award at the University of Extremadura in 2019. He was a recipient of the Outstanding Paper Award in the WHISPERS 2019 Congress. Some of his contributions have been recognized as the hot-topic publication for their impact on the scientific community. From his experience as a reviewer, it is worth mentioning his active collaboration in more than ten scientific journals, such as the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING, IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING, and IEEE GEOSCIENCE AND REMOTE SENSING LETTERS, being awarded the Best Reviewers recognition of the IEEE GEOSCIENCE AND REMOTE SENSING LETTERS in 2018. He has guest-edited three special issues on hyperspectral remote sensing for different journals. He is also an Associate Editor of the IEEE GEOSCIENCE AND REMOTE SENSING LETTERS and IEEE JOURNAL ON MINIATURIZATION FOR AIR AND SPACE SYSTEMS.



Nuno S. Pereira received the degree in physics and the M.Sc. degree in physics from the University of Lisbon, Portugal, in 1993 and 2001, respectively. He is currently pursuing the Ph.D. degree with the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura, Cáceres, Spain.

Since 1996, he has been involved in teaching with the Department of Mathematics and Physical Sciences, Polytechnic Institute of Beja, Portugal, where he has been an Adjunct Professor, since 2003. His research interests include hyperspectral image analysis, and efficient implementation of machine learning algorithms in embedded systems.



Javier Plaza (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees in computer engineering from the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura, Cáceres, Spain, in 2004 and 2008, respectively.

He is also a member of the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura. He has authored more than 150 publications, including over 50 JCR journal articles, ten book chapters, and 90 peer-reviewed conference proceeding papers. His main research interests comprise hyperspectral data processing and parallel computing of remote sensing data.

Dr. Plaza was a recipient of the Outstanding Ph.D. Dissertation Award at the University of Extremadura in 2008. He was a recipient of the Best Column Award of the *IEEE Signal Processing Magazine* in 2015 and the Most Highly Cited Paper (2005–2010) in the *Journal of Parallel and Distributed Computing*. He received best paper awards at the IEEE International Conference on Space Technology and the IEEE Symposium on Signal Processing and Information Technology. He has guest-edited four special issues on hyperspectral remote sensing for different journals. He is also an Associate Editor of the IEEE GEOSCIENCE AND REMOTE SENSING LETTERS and the IEEE Remote Sensing Code Library. More details available at http://www.umbc.edu/rssipl/people/jplaza.



Antonio Plaza (Fellow, IEEE) received the M.Sc. and Ph.D. degrees in computer engineering from the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura, Cáceres, Spain, in 1999 and 2002, respectively.

He is also the Head of the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura. He has authored more than 600 publications, including over 200 JCR journal articles (over 160 in IEEE journals), 23 book chapters, and around 300 peer-reviewed conference proceeding papers. His main research interests comprise hyperspectral data processing and parallel computing of remote sensing data.

Prof. Plaza was also a member of the Steering Committee of the IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING (JSTARS). He is a Fellow of IEEE for contributions to hyperspectral data processing and parallel computing of Earth observation data. He was a recipient of the Best Column Award of the *IEEE Signal Processing Magazine* in 2015, the 2013 Best Paper Award of the JSTARS journal, and the Most Highly Cited Paper (2005–2010) in the *Journal of Parallel and Distributed Computing*. He received best paper awards at the IEEE International Conference on Space Technology and the IEEE Symposium on Signal Processing and Information Technology. He was a recipient of the recognition of Best Reviewers of the IEEE GEOSCIENCE AND REMOTE SENSING LETTERS in 2009 and the recognition of Best Reviewers of the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING in 2010, for which he has served as Associate Editor from 2007 to 2012. He has served as the Director of Education Activities for the IEEE Geoscience and Remote Sensing Society (GRSS) from 2011 to 2012 and the President of the Spanish Chapter of IEEE GRSS from 2012 to 2016. He has guest-edited ten special issues on hyperspectral remote sensing for different journals. He is also an Associate Editor of IEEE ACCESS (receiving recognition as an Outstanding Associate Editor of the journal in 2017). He was a member of the Editorial Board of the IEEE Geoscience and Remote Sensing Newsletter from 2011 to 2012 and the *IEEE Geoscience and Remote Sensing Magazine* in 2013. He has reviewed more than 500 manuscripts for over 50 different journals. He has served as the Editor-in-Chief of the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING from 2013 to 2017. Additional information: http://www.umbc.edu/rssipl/people/aplaza.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.